A project report on

SEMANTIC CACHE FOR LLM SERVING

Submitted in partial fulfillment for the award of the degree of

Bachelor of Technology in

Computer Science and Engineering (Artificial Intelligence and Machine Learning)

by

POJESH KUMAR R (22BAI1373)

VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

April, 2026

VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

DECLARATION

I hereby declare that the thesis entitled "Semantic Cache for LLM Serving" submitted by **Pojesh Kumar R (22BAI1373)**, for the award of the degree of Bachelor of Technology in Computer Science and Engineering, Vellore Institute of Technology, Chennai is a record of bonafide work carried out by me under the supervision of **Dr. Kumar R**.

I further declare that the work reported in this thesis has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Chennai
Date: 06-04-2026

Signature of the Candidate

VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

School of Computer Science and Engineering

CERTIFICATE

This is to certify that the report entitled "**Semantic Cache for LLM Serving**" is prepared and submitted by **Pojesh Kumar R (22BAI1373)** to Vellore Institute of Technology, Chennai, in partial fulfillment of the requirement for the award of the degree of **Bachelor of Technology in Computer Science and Engineering (Artificial Intelligence and Machine Learning)** is a bonafide record carried out under my guidance. The project fulfills the requirements as per the regulations of this University and in my opinion meets the necessary standards for submission. The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma and the same is certified.

Signature of the Guide:
Name: Dr. Kumar R
Date: 06-04-2026

Signature of the Examiner
Name: Dr. Jani Anbarasi L
Date:

Signature of the Examiner
Name: Dr. A.B.Ahadit
Date:

Approved by the Head of Department,
B.Tech CSE (AI & ML)

Name: Dr. Tulasi Prasad Sariki
Date: 06-04-2026

ACKNOWLEDGEMENT

It is my pleasure to express with deep sense of gratitude to **Dr. Kumar R**, School of Computer Science and Engineering, Vellore Institute of Technology, Chennai, for his constant guidance, continual encouragement, understanding; more than all, he taught me patience in my endeavor. My association with him is not confined to academics only, but it is a great opportunity on my part to work with an intellectual and expert in the field of machine learning and distributed systems.

It is with gratitude that I would like to extend my thanks to the visionary leader Dr. G. Viswanathan our Honorable Chancellor, Dr. Sankar Viswanathan, Dr. Sekar Viswanathan, Dr. G.V. Selvam Vice Presidents, Dr. Sandhya Pentareddy, Executive Director, Ms. Kadhambari S. Viswanathan, Assistant Vice-President, Dr. V. S. Kanchana Bhaaskaran Vice-Chancellor, Dr. T. Thyagarajan Pro-Vice Chancellor, VIT Chennai, Dr. K. Sathiyanarayanan, Director, Chennai Campus and Dr. P. K. Manoharan, Additional Registrar for providing an exceptional working environment and inspiring all of us during the tenure of the course.

Special mention to Dr. Viswanathan V, Dean, Dr. Nithyanandam P, Dr. Suganya G, Dr. Sweetlin Hemalatha C, Associate Deans, School of Computer Science and Engineering, Vellore Institute of Technology, Chennai for spending their valuable time and efforts in sharing their knowledge and for helping us in every aspect.

In jubilant state, I express ingeniously my whole-hearted thanks to **Dr. Tulasi Prasad Sariki**, Head of the Department, B.Tech. CSE AIML and the Project Coordinators for their valuable support and encouragement to take up and complete the thesis.

My sincere thanks to all the faculties and staffs at Vellore Institute of Technology, Chennai who helped me acquire the requisite knowledge. I would like to thank my parents for their support. It is indeed a pleasure to thank my friends who encouraged me to take up and complete this task.

Place: Chennai **Pojesh Kumar R**

Date: 06-04-2026

ABSTRACT

Large Language Models now sit at the core of many modern applications, from chatbots and search assistants to content generation tools. Despite their usefulness, running these models at scale is expensive, which makes efficient deployment a real challenge. Studies on around 27,000 ChatGPT conversations show that nearly a third of user queries are semantically similar to ones asked before, pointing to a clear opportunity to reuse past results through caching. Current semantic caching systems like GPTCache and MeanCache rely on fixed similarity thresholds applied across all queries. This one-size-fits-all approach struggles in practice, since different queries need different levels of precision. As a result, these systems either return incorrect cached responses too often or miss chances where caching could have worked effectively.

This project presents a fault-tolerant adaptive semantic cache that addresses these limitations through several key innovations. First, per-entry conformal thresholds replace the single global threshold with individually calibrated similarity boundaries for each cached entry, providing a formal guarantee that the probability of serving an incorrect cached response remains below a user-specified error rate. Second, a Thompson Sampling multi-armed bandit with Beta posteriors explores the threshold parameter space to learn optimal default thresholds for newly cached entries. Third, an ADWIN-based drift detector monitors the distribution of cache hit rewards and resets the bandit posteriors when a statistically significant shift is detected. Fourth, a Semantic W-TinyLFU admission policy uses locality-sensitive hashing over Count-Min Sketch frequency estimates to reject one-hit-wonder queries that would pollute the cache. Fifth, CostGDSF eviction prioritizes retention of entries that are expensive to regenerate via the LLM. The system employs a two-tier lookup architecture combining $O(1)$ exact-hash matching with $O(\log n)$ FAISS HNSW approximate nearest-neighbor search, with Redis backing for distributed deployments.

Evaluation on the Natural Questions dataset with 650 queries including 150 paraphrase variants demonstrates that the proposed system achieves a hit rate of 20.8% and Precision of 62.2%, outperforming GPTCache (19.3% hit rate, 60.9 Precision), vCache (18.6% hit), and four other baseline strategies. The system saves 135 LLM inference calls out of 650 queries while maintaining competitive precision at 62.22%. A full-stack implementation is provided comprising a Python FastAPI backend and a Tauri/SvelteKit analytics dashboard with real-time monitoring, side-by-side cache comparison, and an integrated benchmark suite.

Contents

List of Abbreviations **vii**

1 Introduction **1**
1.1 Background 1
1.2 The Query Redundancy Opportunity 1
1.3 Limitations of Existing Approaches 2
1.4 Problem Statement 2
1.5 Objectives 2
1.6 Scope of the Project 3
1.7 Organization of the Report 4

2 Literature Review **5**
2.1 Introduction 5
2.2 Existing Semantic Cache Systems 5
2.3 Related Algorithmic Foundations 6
2.4 Literature Survey Summary 7
2.5 Research Gap 8

3 System Design **9**
3.1 High-Level Architecture 9
3.2 Query Pipeline Complexity 10
3.3 Cache Management Components 11
3.4 Two-Tier Lookup Architecture 11
3.5 Per-Entry Conformal Thresholds 11
3.6 Thompson Sampling Bandit for Threshold Exploration 13
3.7 ADWIN Drift Detection 14
3.8 Semantic W-TinyLFU Admission Policy 14
3.9 CostGDSF Eviction 14
3.10 Session Context Fusion 14
3.11 Technology Stack 15
3.12 Backend Module Structure 15
3.13 API Endpoints 17

3.14 Frontend Dashboard . . . 17
3.15 Benchmark Framework . . . 18
3.16 Testing . . . 18
3.17 Distributed Systems Design . . . 19
3.18 Configuration Parameters . . . 21

4 Results and Discussion **22**
4.1 Experimental Setup . . . 22
4.2 Results on Natural Questions . . . 23
4.3 Results on SQuAD . . . 24
4.4 Precision Analysis . . . 25
4.5 Latency Analysis . . . 25
4.6 Thompson Sampling Arm Distribution . . . 26
4.7 Cross-Dataset Comparison . . . 27
4.8 Ablation Analysis . . . 27
4.9 Cost Savings Estimation . . . 28
4.10 False Positive Analysis . . . 29
4.11 Implementation Demonstration . . . 29

5 Challenges and Solutions **33**
5.1 Conformal Threshold Cold Start . . . 33
5.2 Embedding Model Selection . . . 33
5.3 FAISS Memory Management . . . 33
5.4 Admission Policy Calibration . . . 33
5.5 LLM Conversation Context . . . 34
5.6 Benchmark Fairness and Shared Resources . . . 34
5.7 Text Normalization Consistency . . . 34
5.8 Circuit Breaker for External Dependencies . . . 35
5.9 Count-Min Sketch Frequency Aging . . . 35

6 Conclusion **36**
6.1 Conclusion . . . 36
6.2 Future work . . . 37

References **38**

Appendices **40**
Appendix A: API Request and Response Formats . . . 40
Appendix B: Figures . . . 41
Appendix C: Tables . . . 45

Appendix D: Algorithms 51

List of Tables

2.1 Literature Survey Summary 7

3.1 Query Pipeline Complexity Analysis 10
3.2 Cache Management Component Summary 11
3.3 Thompson Sampling Threshold Arms Configuration 13
3.4 Technology Stack 15
3.5 API Endpoint Summary 17
3.6 Configuration Parameters 21

4.1 Benchmark Results on Natural Questions Dataset (650 queries, 300 capacity) 23
4.2 Benchmark Results on SQuAD Dataset (650 queries, 300 capacity) . . . 24
4.3 Precision Analysis on NQ — Top Three Strategies 25
4.4 Latency Comparison on NQ 25

List of Figures

3.1 System Architecture Diagram 9

4.1 Hit Rate and Precision Comparison across Strategies (NQ Dataset) . . . 23
4.2 Latency Analysis across Strategies (Log Scale) 26
4.3 Chat Interface demonstrating cache hit/miss behavior 30
4.4 Comparison view showing cached vs direct LLM response 30
4.5 Analytics dashboard with real-time monitoring 31
4.6 Benchmark suite with dataset selection and results 32

LIST OF ABBREVIATIONS

Abbreviation	Full Form
LLM	Large Language Model
NQ	Natural Questions
ANN	Approximate Nearest Neighbor
HNSW	Hierarchical Navigable Small World
FAISS	Facebook AI Similarity Search
CMS	Count-Min Sketch
LSH	Locality-Sensitive Hashing
GDSF	Greedy Dual Size Frequency
W-TinyLFU	Windowed Tiny Least Frequently Used
ADWIN	Adaptive Windowing
EMA	Exponential Moving Average
MAB	Multi-Armed Bandit
API	Application Programming Interface
TTL	Time To Live
SLRU	Segmented Least Recently Used
FedAvg	Federated Averaging
WAL	Write-Ahead Log

CHAPTER 1
Introduction

1.1 Background

Models like GPT-4, Claude, LLaMA, and Gemini have ended up becoming the backbone of a lot of modern AI applications. You see them everywhere now—chatbots, coding tools, search, content generation—you name it. As more companies plug them into their products, the number of requests being handled daily has shot up to massive scales.

The catch is that running all of this isn't cheap. Every query eats up compute, usually on GPU-heavy infrastructure, and most providers bill based on token usage. Once traffic starts increasing, the costs don't just grow—they ramp up fast. It's not unusual for even fairly standard deployments to rack up thousands of dollars per day just to keep inference running.

Then there's latency, which is just as tricky to deal with. Responses can come back quickly or take a few seconds depending on the model and input size, but either way, users notice. For things like chatbots or search, people expect replies almost instantly. Even small delays start to feel frustrating, and that directly affects how much they trust or continue using the system.

1.2 The Query Redundancy Opportunity

Studies looking at how people actually use large language models have pointed out a pretty clear opportunity to optimize things. In one analysis of around 27,000 ChatGPT conversations, roughly 31% of the queries turned out to be semantically similar to something that had already been asked. This happens because users often ask the same question in slightly different ways—changing wording, sentence structure, or how detailed the question is.

The problem is that traditional caching methods don't really capture this. They rely on exact matches so even some small changes in phrasing will make two identical questions look completely different. Example statements like "What is the capital of France?" and "Tell me the capital city of France" are same in terms of meaning, but string matching doesn't recognize this.

Semantic caching tries to solve this by looking beyond the surface text. Instead of matching words directly, it converts queries into vector representations in an embedding space. In embedding space similar meaning questions end up close to each other that makes it viable to reuse previous answers even though the queries are not exact matches.

1.3 Limitations of Existing Approaches

Several semantic caching systems have been introduced in recent work, such as GPT-Cache, MeanCache, SCALM, and GPT Semantic Cache. While they show that semantic caching can work in practice, they still run into a few core limitations that are hard to ignore.

The most significant limitation is the reliance on a single global similarity threshold. GPTCache uses a fixed threshold of 0.75 to decide whether a cached response is sufficiently similar to a new query. MeanCache computes a threshold as the mean minus one standard deviation of observed similarities. In both cases, the same threshold is applied across all cached entries, without considering their individual differences. This kind of one-size-fits-all approach doesn't really hold up in practice, since different queries often need different levels of matching precision. A factual question about a specific date or number demands a very high similarity match to avoid serving incorrect information, while an open-ended question about a broad topic can tolerate lower similarity scores.

Additionally, existing systems employ basic cache management strategies. Most use simple LRU eviction that treats all cached entries as equally valuable, ignoring the fact that some entries are far more expensive to regenerate than others. None of the existing systems incorporate intelligent admission policies that can filter out rare queries unlikely to be re-queried.

1.4 Problem Statement

The central problem addressed by this project is the design and implementation of a semantic cache for LLM serving that overcomes the limitations of fixed global thresholds through per-entry adaptive threshold learning, while incorporating cost-aware cache management and providing formal guarantees on the probability of serving incorrect cached responses.

1.5 Objectives

The specific objectives of this project are as follows:

1. Design and implement per-entry conformal similarity thresholds that learn from correctness feedback, providing a formal guarantee that the probability of a false positive (incorrect cache hit) remains below a user-specified error rate of 5 percent.

2. Implement a Thompson Sampling multi-armed bandit with Beta posteriors to explore the threshold parameter space and learn optimal default thresh-

olds for newly cached entries, coupled with ADWIN drift detection to handle distribution shifts.

3. Develop a Semantic W-TinyLFU admission policy that uses locality-sensitive hashing over Count-Min Sketch frequency estimates to reject infrequent queries that would pollute the cache.

4. Implement CostGDSF eviction that prioritizes retention of cached entries that are expensive to regenerate, using a priority function that incorporates frequency of access, regeneration cost, and storage size.

5. Build a two-tier lookup architecture combining $O(1)$ exact-hash matching with $O(\log n)$ FAISS HNSW approximate nearest-neighbor search, with optional Redis backing for distributed exact-match lookups.

6. Develop a comprehensive benchmark framework comparing the proposed system against six baseline strategies on public QA datasets.

7. Build a full-stack analytics dashboard for real-time monitoring, cache comparison, and benchmark visualization.

1.6 Scope of the Project

The scope of this project encompasses the complete design, implementation, and evaluation of the semantic cache system. On the backend side, the system supports three LLM providers: Ollama for local GPU-accelerated inference, Groq for cloud-based low-latency inference, and the OpenAI API for broader model access. The choice of provider is configurable at runtime through environment variables, allowing users to switch between local and cloud backends without modifying any code. The embedding pipeline uses the paraphrase-MiniLM-L6-v2 model from Sentence Transformers, producing 384-dimensional dense vectors that serve as the basis for all semantic similarity computations.

For evaluation, two well-established public question-answering datasets from HuggingFace were selected: the Natural Questions dataset compiled by Google from real web search queries, and the Stanford Question Answering Dataset which contains reading comprehension questions derived from Wikipedia articles. These two datasets were chosen because they represent different styles of questioning and answer formats, providing a more rounded picture of how the cache performs across varied workloads.

The frontend analytics dashboard is implemented as a cross-platform desktop application using the Tauri framework with a Rust-based backend shell and a SvelteKit frontend written in Svelte 5. This combination was chosen for its small binary size, low

memory footprint, and native performance compared to Electron-based alternatives. The dashboard communicates with the Python backend exclusively through REST API calls, maintaining a clean separation between the cache logic and the visualization layer.

The system also supports multi-turn conversational context through session-aware query embedding fusion. Each active session accumulates a running context vector that blends prior conversation turns with the current query, allowing the cache to recognize that a follow-up question like "and what about its population?" relates to a previously discussed city even though the follow-up query alone carries insufficient context.

Distributed deployment components including consistent hashing with virtual nodes, a write-ahead log with CRC32 integrity checks, vector clocks for causal ordering, and a push-pull gossip protocol for bandit posterior synchronization are fully implemented and unit-tested. However, multi-node integration testing and deployment evaluation fall outside the scope of this report and are discussed as future work.

1.7 Organization of the Report

The remainder of this report is organized as follows. Chapter 2 presents a comprehensive review of existing semantic cache systems and the algorithmic foundations that underpin the proposed design, concluding with an identification of the research gaps addressed by this work. Chapter 3 describes the system design in detail, covering the two-tier lookup architecture, per-entry conformal thresholds, Thompson Sampling bandit, ADWIN drift detection, admission and eviction policies, the distributed systems components, the frontend dashboard, the benchmark framework, and the testing strategy. Chapter 4 presents the experimental results on both the Natural Questions and SQuAD datasets, with detailed analysis of hit rates, precision, latency, cost savings, and cross-dataset comparisons. Chapter 5 discusses the key challenges encountered during development and the solutions adopted to address them. Chapter 6 concludes the report with a summary of contributions and directions for future work.

CHAPTER 2
Literature Review

2.1 Introduction

Semantic caching for LLM-based systems has quickly become an active area of research over the past couple of years, mainly because of how fast these applications have grown and how expensive inference has become. This chapter takes a closer look at the approaches that already exist, points out where they fall short, and places the proposed system in the context of the broader research landscape.

2.2 Existing Semantic Cache Systems

GPTCache (Regmi and Pun, 2024): GPTCache is a Redis-based semantic caching system that uses HNSW approximate nearest-neighbor search to find similar queries in the cache. It employs a fixed similarity threshold, typically set to 0.75, and uses LRU eviction for cache management. While GPTCache shows that semantic caching can help reduce LLM costs, its use of a fixed threshold makes performance uneven across different types of queries. In our experiments on the Natural Questions dataset, it reached a hit rate of 19.38% with a precision of 61.9%, which means that a noticeable portion of its cache hits close to 38% ended up returning incorrect responses.

MeanCache (Gill et al., 2024): MeanCache introduces user-centric semantic caching with federated learning for similarity model training. It computes an adaptive threshold as the mean minus one standard deviation of observed similarity scores and uses PCA-based embedding compression. While MeanCache's threshold adaptation represents an improvement over static approaches, it operates on a global level and cannot distinguish between query types that require different precision levels. MeanCache achieves 13.23% hit rate with 66.28% precision in our evaluation.

vCache (Schroeder et al., 2025): vCache is the most closely related work to our system. It introduces per-entry conformal thresholds using online conformal prediction, providing formal guarantees on the false positive rate. However, vCache uses a higher default threshold of 0.80, employs simple LRU eviction, and lacks admission control. Our system extends the conformal threshold approach with Thompson Sampling exploration for learning optimal default thresholds, CostGDSF eviction for value-aware cache management, and W-TinyLFU admission for frequency-gated entry filtering.

RAGCache (Jin et al., 2024): RAGCache focuses on caching for retrieval-augmented generation workloads. It uses a higher similarity threshold of 0.85 and incorporates cost-aware GDSF eviction similar to our approach. However, RAGCache does not im-

plement per-entry adaptive thresholds and uses a static global threshold.

SCALM (Li et al., 2024): SCALM targets semantic caching for automated chat services with vector quantization and semantic clustering. It achieves significant token savings but does not include adaptive threshold mechanisms.

MinCache (Haqiq et al., 2025): MinCache implements a three-tier hierarchy using exact, resemblance, and semantic matching. It achieves 4.5 times speedup over GPTCache but relies on MinHash approximation that misses nuanced semantic relationships.

2.3 Related Algorithmic Foundations

Conformal Prediction (Vovk et al., 2005): Conformal prediction provides a framework for constructing prediction sets with guaranteed coverage. Given a sequence of observations, conformal prediction computes a nonconformity score for each new example and uses the quantile of historical scores to set a threshold that guarantees the desired error rate. Our per-entry conformal thresholds apply this framework to the cache hit decision, treating each cached entry as an independent prediction problem.

Thompson Sampling (Thompson, 1933): Thompson Sampling is a Bayesian approach to multi armed bandit problem. Every arm maintains a Beta posterior distribution with count of successes (α) and failures (β) as parameters. A sample is drawn from each arm's posterior in each trial and the arm with highest sample is selected. This gives natural exploration exploitation balance without needing explicit exploration parameters.

ADWIN Drift Detection (Bifet and Gavalda, 2007): ADWIN maintains a variable-length sliding window of recent observations and detects distribution changes by comparing sub-windows using Hoeffding bounds. When a statistically significant change is detected, the window is shrunk to contain only post-change data. We use ADWIN to detect shifts in the cache hit reward distribution, triggering bandit posterior resets when the query distribution changes.

W-TinyLFU (Einziger et al., 2017): The Windowed Tiny Least Frequently Used admission policy combines a small window cache that admits all entries with a frequency-gated main cache. New entries must demonstrate sufficient access frequency to displace existing entries from the window. Frequency is estimated using a Count-Min Sketch, providing space-efficient approximate counting. Our Semantic W-TinyLFU extends this with LSH-bucketed frequency estimation, where semantically similar queries contribute to the same frequency counter.

GDSF (Young, 2000): Greedy Dual Size Frequency eviction policy assign a priority score for each cache entry by its access frequency, regeneration cost, and storage size. Entry with least priority is evicted first. An inflation factor is updated on each

eviction to prevent recently inserted entries from being immediately evicted.

FedAvg (McMahan et al., 2017): Federated Averaging enables distributed training of machine learning models by averaging locally trained model weights across multiple clients without sharing raw data. Our system implements FedAvg-based synchronization of Thompson Sampling bandit posteriors across distributed cache nodes.

2.4 Literature Survey Summary

Table 2.1 presents a summary of the key works reviewed in this chapter, highlighting their approaches, contributions, and identified gaps.

Table 2.1: Literature Survey Summary

S.No.	Author	Approach	Key Contribution	Gap Identified
1	Haqiq et al., 2025	MinCache: Hybrid Cache	Three-tier hierarchy; 4.5× speedup vs GPTCache	MinHash misses nuanced semantics
2	Gill et al., 2024	MeanCache: User-Centric Caching	Federated learning for similarity models	No adaptive thresholds; fixed metrics
3	Gao et al., 2025	Online Context Caching	KV cache placement + request scheduling	Does not address semantic similarity
4	Li et al., 2024	SCALM: Semantic Caching	Vector quantization; semantic clustering	Semantic analysis overhead; no adaptation
5	Li et al., 2024	Context-based Semantic Caching	Context embedding + attention weighting	Increased memory footprint
6	Liu et al., 2024	Lost in the Middle	Position bias analysis in transformers	Affects compression, not caching
7	Regmi & Pun, 2024	GPT Semantic Cache	Redis-based with HNSW ANN	Fixed threshold; lacks adaptability
8	Jin et al., 2024	RAGCache	Cost-aware GDSF eviction	No per-entry threshold adaptation
9	Schroeder et al., 2025	vCache	Per-entry conformal thresholds	High default; no bandit exploration
10	Zha et al., 2023	Online Optimization for Cache	Bayesian optimization for thresholds	Single query type; manual init

S.No.	Author	Approach	Key Contribution	Gap Identified
11	Zhu et al., 2023	Semantic Caching for LLM Cost	Embedding-based lookup	Fixed threshold τ=0.8; no adaptation
12	Einziger et al., 2017	TinyLFU	Frequency-gated admission	Not designed for semantic caching
13	Bifet & Gavalda, 2007	ADWIN	Adaptive windowing for drift	General-purpose; needs integration
14	Vovk et al., 2005	Conformal Prediction	Guaranteed prediction sets	Theoretical; per-entry adaptation needed
15	McMahan et al., 2017	FedAvg	Federated model averaging	Not applied to cache threshold learning

2.5 Research Gap

The review of existing literature reveals three principal gaps:

1. **No per-entry threshold adaptation:** All existing systems except vCache apply a single global threshold. Even vCache uses a relatively high default (0.80) and lacks exploration mechanisms for learning optimal thresholds.

2. **Primitive cache management:** Most systems use simple LRU eviction and admit all entries unconditionally, ignoring the heterogeneous value of cached entries in terms of regeneration cost and access frequency.

3. **No formal correctness guarantees with adaptive thresholds:** While vCache provides conformal guarantees, it does not combine them with bandit-based exploration or drift-aware threshold resetting.

The proposed system addresses all three gaps through an integrated approach combining conformal prediction, Thompson Sampling exploration, ADWIN drift detection, W-TinyLFU admission, and CostGDSF eviction.

CHAPTER 3
System Design

3.1 High-Level Architecture

The system follows a layered architecture with clear separation of concerns. At the highest level, it consists of two main components: a Python FastAPI backend that implements the cache logic, LLM integration, and REST API, and a Tauri/SvelteKit frontend that provides the analytics dashboard, chat interface, and benchmark visualization.

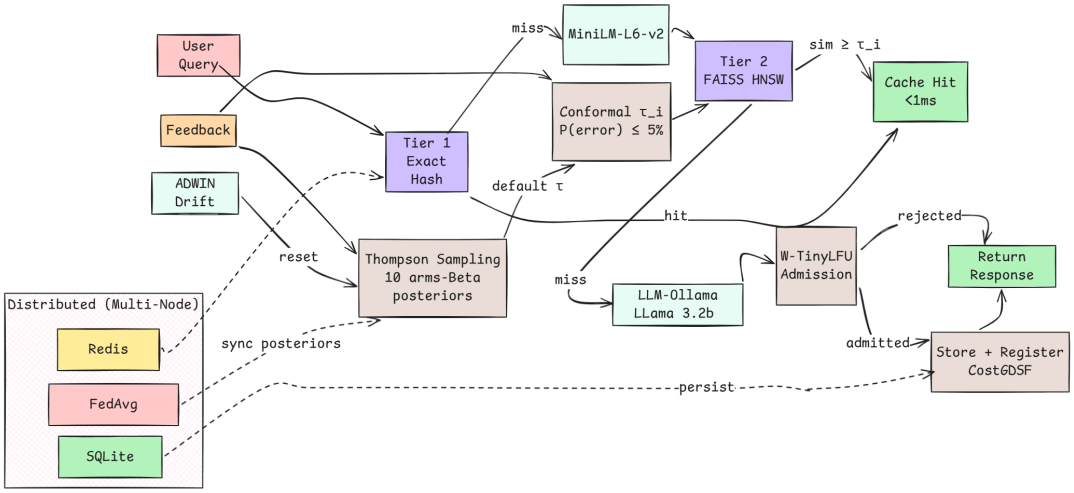

Figure 3.1: System Architecture Diagram

Figure 3.1 illustrates the complete data flow through the ContextualCache system. A user query enters from the top left and first passes through the Tier 1 exact hash lookup. On a miss, the query is encoded using the MiniLM-L6-v2 embedding model and forwarded to the Tier 2 FAISS HNSW index for semantic search. If a candidate exceeds its per-entry conformal threshold, the cached response is returned. Otherwise, the query falls through to the LLM backend, and the generated response passes through the W-TinyLFU admission gate before being stored with CostGDSF eviction metadata. The diagram also shows the feedback loop where correctness signals flow back to the conformal threshold calibrator, the Thompson Sampling bandit that learns optimal default thresholds, and the ADWIN drift detector that triggers bandit resets. On the left side, the distributed components are depicted: Redis for shared exact-match storage, FedAvg for bandit posterior synchronization, and SQLite for local persistence.

The central orchestrator is the ContextualCacheManager which coordinates all cache operations. When a query arrives, it flows through the following pipeline:

1. **Tier 1 Exact Hash Lookup:** The query text is normalized (lowercased, punctuation stripped, whitespace collapsed) and hashed using SHA-256.

The hash is checked against an in-memory dictionary and optionally against a Redis backing store. If found, the cached response is returned immediately with sub-millisecond latency.

2. **Embedding Generation:** If the exact hash lookup misses, the query is encoded into a 384-dimensional dense vector using the paraphrase-MiniLM-L6-v2 sentence transformer model. An LRU embedding cache (10,000 entries) avoids redundant encoding of recently seen queries.

3. **Session Context Fusion:** For multi-turn conversations, the query embedding is fused with the session context vector using exponential moving average with a query-dominant weight of $\alpha = 0.85$.

4. **Tier 2 Semantic Search:** The fused embedding is used to query a FAISS HNSW index that returns the top-k (default $k = 10$) nearest neighbors. For each candidate, the cosine similarity is computed and compared against the per-entry conformal threshold.

5. **Cache Hit or Miss Decision:** If any candidate exceeds its per-entry threshold, the cached response is returned. Otherwise, the query is forwarded to the LLM provider.

6. **LLM Fallback:** On cache miss, the query is sent to the configured LLM backend (Ollama, Groq, or OpenAI). The response is returned to the user and simultaneously processed for cache admission.

7. **Admission and Storage:** The new entry passes through the W-TinyLFU admission policy. If admitted, it is stored in both tiers with CostGDSF eviction metadata.

3.2 Query Pipeline Complexity

Table 3.1: Query Pipeline Complexity Analysis

Stage	**Component**	**Complexity**	**Latency**
Tier 1	Exact Hash Lookup (SHA-256)	$O(1)$	<1 ms
Tier 2	FAISS HNSW Semantic Search	$O(\log n)$	2–15 ms
Embedding	paraphrase-MiniLM-L6-v2	$O(d)$	5–20 ms
Threshold	Per-entry conformal τ_i	$O(k \log k)$	<0.1 ms
Miss	LLM Provider (Ollama/Groq)	Network	500–6000 ms

3.3 Cache Management Components

Table 3.2: Cache Management Component Summary

Component	Algorithm	Purpose
Admission	Semantic W-TinyLFU	Reject one-hit-wonders based on LSH-bucketed frequency
Eviction	CostGDSF	$H = f \times c/s + L$ (keeps expensive entries)
Threshold Learning	Thompson Sampling (10 arms)	Learn optimal default τ for new entries
Drift Detection	ADWIN	Reset bandit posteriors on distribution shift
Session Context	EMA fusion (α=0.85)	Multi-turn conversational awareness
Fault Isolation	Circuit Breaker	Per-dependency isolation (CLOSED/OPEN/HALF_OPEN)

3.4 Two-Tier Lookup Architecture

The two-tier lookup architecture is designed to minimize the average query latency. Tier 1 handles exact matches without requiring any embedding computation, saving approximately 5–20 milliseconds per query. Only on a Tier 1 miss does the system invoke the embedding model and FAISS index for Tier 2 semantic search.

The FAISS index uses the HNSW algorithm with the following parameters: $M = 32$ links per node, efConstruction $= 200$ for index build quality, and efSearch $= 64$ for query-time accuracy. The index is wrapped in IndexIDMap2 for explicit ID management, allowing true vector removal. A periodic rebuild is triggered after every 500 removals to reclaim fragmented space.

3.5 Per-Entry Conformal Thresholds

The cornerstone of the proposed system is the per-entry conformal threshold mechanism. Unlike global-threshold approaches where a single value governs all cache hit decisions, our system maintains an individually calibrated threshold for each cached entry.

Formulation: For a cached entry with identifier i, the conformal threshold τ_i is computed as the $(1 - \varepsilon)$ quantile of the entry's nonconformity scores, where ε is the

target error rate (default 0.05). The nonconformity score for a query against entry i is defined as:

For a correct cache hit with similarity s:

$$\text{nonconformity} = 1 - s \tag{3.1}$$

For an incorrect cache hit with similarity s:

$$\text{nonconformity} = \min(1.0, \max(1 - s + 0.1, 0.5)) \tag{3.2}$$

Calibration Process: New entries start with a default threshold of 0.70. After accumulating at least 3 feedback signals, the threshold transitions to the conformal quantile computation. A sliding window of the most recent 200 nonconformity scores ensures adaptation to changing query patterns. The threshold is clamped to the range $[0.60, 0.99]$ to prevent extreme values.

Formal Guarantee: Under the exchangeability assumption of conformal prediction, the probability of a false positive satisfies:

$$P(\text{false positive}) \leq \varepsilon \tag{3.3}$$

Algorithm 1: Conformal Threshold Update

```
Input: entry_id, similarity, was_correct
If was_correct:
    nonconformity = 1.0 - similarity
Else:
    nonconformity = min(1.0, max(1.0 - similarity + 0.1, 0.5))

scores[entry_id].append(nonconformity)
If len(scores) > max_window:
    scores = scores[-max_window:]

If len(scores) >= min_calibration_points:
    sorted_scores = sort(scores)
    idx = ceil((1 - epsilon) * (len + 1)) - 1
    tau_i = 1.0 - sorted_scores[idx]
    tau_i = clamp(tau_i, 0.60, 0.99)
Else:
    tau_i = default_threshold
```

3.6 Thompson Sampling Bandit for Threshold Exploration

While conformal prediction provides per-entry fine-tuning after sufficient feedback, new entries initially use a default threshold. The Thompson Sampling bandit learns what this default should be by exploring different threshold values and observing their outcomes.

The bandit maintains 10 arms corresponding to threshold values linearly spaced from 0.65 to 0.95. Each arm i has a Beta posterior α_i (successes) and β_i (failures) params both init to 1.

Table 3.3: Thompson Sampling Threshold Arms Configuration

Arm	Threshold Value
0	0.650
1	0.683
2	0.717
3	0.750
4	0.783
5	0.817
6	0.850
7	0.883
8	0.917
9	0.950

Algorithm 2: Thompson Sampling Update

```
On each cache hit:
    arm* = current best arm (highest expected reward)
    If similarity >= 0.85: reward = 1.0
    Else if similarity < 0.65: reward = 0.0
    Else: reward = (similarity - 0.65) / 0.20

    alpha[arm*] += reward
    beta[arm*] += (1.0 - reward)

    Feed reward to ADWIN drift detector
    If drift detected:
        Reset all alpha, beta to (2, 2)
```

The expected reward for each arm is computed as $\alpha_i/(\alpha_i + \beta_i)$, representing the empirical success rate. The arm with the highest expected reward determines the current best default threshold.

3.7 ADWIN Drift Detection

The ADWIN (Adaptive Windowing) algorithm monitors the stream of bandit rewards for distribution changes. It maintains a variable-length window of recent observations and uses Hoeffding bounds to test whether the means of two sub-windows differ significantly. When a drift is detected, the bandit posteriors are reset to weakly informative priors ($\alpha = 2, \beta = 2$), forcing renewed exploration.

3.8 Semantic W-TinyLFU Admission Policy

The admission policy determines whether a new entry should be admitted to the cache on a miss. The W-TinyLFU design consists of two stages:

1. **Window Cache (5% of capacity):** Admits all entries freely, providing a cold-start period where new entries can accumulate frequency without being rejected.
2. **Main Cache (95% of capacity):** Requires that a new entry's frequency exceeds that of the window's LRU victim to gain admission.

Semantic Extension: Rather than tracking frequency by exact query text, our system uses Random Projection LSH (8 bits, 4 tables) to map embedding vectors to discrete buckets. Semantically similar queries map to the same bucket, sharing a frequency counter in the Count-Min Sketch (width=4096, depth=4).

3.9 CostGDSF Eviction

When the cache reaches capacity and a new entry must be admitted, the CostGDSF policy selects the entry with the lowest priority score for eviction:

$$H(e) = \frac{\text{frequency}(e) \times \text{regen_cost}(e)}{\text{storage_bytes}(e)} + L \tag{3.4}$$

where $\text{regen_cost}(e) = (\text{llm_cost_usd} \times 10^6) + \text{embed_cost_ms}$ and L is an inflation factor updated to the priority of the last evicted entry.

3.10 Session Context Fusion

For multi-turn conversations, the system maintains a session context accumulator. When a new query arrives within an active session, its embedding is fused with the accumulated context vector using exponential moving average:

$$\mathbf{c}_{\text{new}} = \alpha \cdot \mathbf{q} + (1 - \alpha) \cdot \mathbf{c}_{\text{old}} \tag{3.5}$$

where $\alpha = 0.85$ (query-dominant). The fused vector is normalized to unit length before FAISS search. Sessions expire after 30 minutes of inactivity.

3.11 Technology Stack

Table 3.4: Technology Stack

Category	**Technology**
Language	Python 3.10
Backend Framework	FastAPI with Uvicorn ASGI
Embedding Model	paraphrase-MiniLM-L6-v2 (384d)
Vector Index	FAISS (faiss-cpu) with HNSW
LLM Backend	Ollama (LLaMA 3.2:3b) / Groq / OpenAI
Optional Cache Store	Redis (redis-py async)
Persistence	SQLite
Frontend Framework	SvelteKit (Svelte 5)
Desktop Runtime	Tauri (Rust)
Charting	Chart.js
Deep Learning	PyTorch (CUDA 12.6)
Testing	pytest with pytest-asyncio (120 tests)

3.12 Backend Module Structure

The backend is organized into the following Python modules within the `contextual_cache` package:

- **cache_manager.py** — Central orchestrator coordinating all cache operations, query pipeline, feedback processing, and statistics aggregation.
- **lookup_engine.py** — Two-tier lookup with SHA-256 exact hash and FAISS IndexIDMap2(IndexHNSWFlat).
- **embedding_service.py** — Sentence-transformers wrapper with LRU embedding cache.
- **conformal_thresholds.py** — Per-entry conformal threshold store with online calibration.

- **bandit.py** — Thompson Sampling MAB with Beta posteriors and ADWIN integration.
- **admission_policy.py** — Semantic W-TinyLFU with LSH-bucketed CMS frequency estimation.
- **eviction.py** — CostGDSF eviction with inflation factor tracking.
- **drift_detection.py** — ADWIN drift detector using Hoeffding bounds.
- **circuit_breaker.py** — Per-dependency circuit breaker (CLOSED/OPEN/HALF_OPEN).
- **session_context.py** — Session context accumulator with EMA fusion.
- **llm_provider.py** — Unified LLM abstraction over Ollama, Groq, and OpenAI.
- **redis_store.py** — Optional Redis-backed exact-match store with circuit breaker.
- **persistence.py** — SQLite persistence for cache entries, scores, bandit state, and chat history.
- **metrics.py** — Metrics collector powering the dashboard.
- **data_structures.py** — CMS, LSH, LRU, and SLRU implementations.
- **config.py** — Pydantic BaseSettings with CC_ environment variable overrides.

3.13 API Endpoints

Table 3.5: API Endpoint Summary

Method	Path	Description
POST	/query	Main cache-fronted LLM query
POST	/feedback	Correctness feedback for calibration
POST	/bulk-query	Batch queries
POST	/api/query-direct	Bypass cache, call LLM directly
GET	/health	Health check
GET	/api/stats	Full analytics for dashboard
POST	/api/clear-cache	Clear all cache entries
GET	/api/conversations	List chat sessions
POST	/api/benchmark/run	Start benchmark run
GET	/api/benchmark/status/{id}	Poll benchmark progress
GET	/api/benchmark/results	List benchmark results

3.14 Frontend Dashboard

The frontend is built as a cross-platform desktop application using Tauri (Rust runtime) with SvelteKit (Svelte 5) and Chart.js for visualization. It features a dark-themed interface with four main pages:

Chat Interface: A conversational interface with session management, cache hit/miss metadata display, token usage tracking, and a collapsible conversation history sidebar. Sessions are persisted via the backend SQLite database.

Cache Comparison View: A side-by-side comparison tool that simultaneously queries the cache and the LLM directly, displaying latency, token count, and speedup metrics.

Analytics Dashboard: A real-time analytics page displaying six key performance indicators, hit rate trends, query tier breakdown, latency time series, Thompson Sampling arm distribution, and similarity distribution. The dashboard polls the backend every three seconds.

Benchmark Suite: An integrated benchmark runner with configurable parameters (number of questions, cache capacity, dataset selection between NQ and SQuAD), real-time progress tracking, and results visualization.

Screenshots of all four pages are presented in Section 4.7.

3.15 Benchmark Framework

The benchmark system compares ContextualCache against six baseline strategies under identical conditions:

1. **Exact-Match LRU** — Hash-based exact matching with LRU eviction (lower-bound baseline).
2. **GPTCache** — Static threshold (0.75) with LRU eviction [3].
3. **MeanCache** — Adaptive threshold (mean − std) with PCA compression and LRU [2].
4. **vCache** — Per-entry conformal thresholds with LRU eviction [4].
5. **RAGCache** — Static threshold (0.85) with cost-aware GDSF eviction [8].
6. **No-Admission** — Conformal thresholds with CostGDSF but no admission gate (ablation study).

All strategies receive identical pre-computed embeddings and LLM responses to ensure fair comparison.

3.16 Testing

The system includes 120 automated tests organized across 12 test files, all executed through pytest with the pytest-asyncio plugin running in auto mode so that asynchronous test functions are handled transparently. The test suite was designed to cover every major component of the system in isolation, ensuring that individual modules behave correctly before they are wired together in the cache manager.

The data structures tests in `test_data_structures.py` verify the Count-Min Sketch implementation including conservative updates, frequency halving for aging, and the property that estimates never undercount. The same file tests the Random Projection LSH hash function for determinism and collision properties, and validates both the LRU and Segmented LRU eviction data structures for correct ordering and capacity enforcement.

The session context tests verify that the EMA fusion formula produces the expected blended vectors when given known inputs, that session expiration occurs after the configured inactivity timeout, and that creating a new session produces a fresh context vector uncontaminated by prior sessions.

Conformal threshold tests check the core calibration logic: that the threshold converges toward the correct quantile as feedback accumulates, that the clamping bounds of

0.60 and 0.99 are respected even when the raw quantile falls outside that range, and that entries with fewer than the minimum calibration points fall back to the default threshold rather than producing unstable estimates.

The eviction and admission tests exercise CostGDSF priority computation with known frequency, cost, and size values to verify the priority formula, and test that W-TinyLFU correctly rejects low-frequency candidates while admitting high-frequency ones. The fault tolerance tests cover the circuit breaker state machine transitions between CLOSED, OPEN, and HALF_OPEN states, ADWIN drift detection with synthetic distribution shifts, and Thompson Sampling arm selection with controlled reward sequences.

The lookup engine tests are among the most critical, verifying that stored entries can be retrieved by both exact hash and semantic search, that FAISS ID mapping is consistent after insertions and deletions, that the periodic index rebuild produces correct results, and that text normalization is applied consistently so that "What is AI?" and "what is ai" map to the same hash. TTL expiration is also tested by inserting entries with short time-to-live values and confirming they are excluded from search results after expiration.

Persistence tests round-trip cache entries through the SQLite layer, checking that embeddings survive serialization with full floating-point fidelity, that bandit posteriors and conformal scores are correctly saved and restored, and that conversation history can be created, queried, and deleted without data corruption.

The remaining test files cover configuration validation (rejecting invalid capacity or threshold values), rate limiter token bucket behavior and per-tenant isolation, consistent hash ring distribution and node addition/removal stability, vector clock ordering and merge semantics, and write-ahead log append/replay with CRC integrity verification and checkpoint-based LSN recovery.

All tests run without network access or GPU requirements, using lightweight in-memory instances of each component. The full suite completes in under 15 seconds on a standard development machine.

3.17 Distributed Systems Design

Although the primary evaluation in this report focuses on a single-node deployment, the system includes a complete set of distributed systems components designed for future multi-node scaling. These components were developed and unit-tested alongside the core cache logic to ensure architectural readiness for horizontal scaling.

The consistent hashing module implements a SHA-256-based hash ring with configurable virtual nodes (default 150 per physical node). Virtual nodes spread each physical node's responsibility across many points on the ring, reducing the variance in load dis-

tribution. When a node is added or removed, only the keys that fall in the affected arc of the ring need to be reassigned, minimizing data movement. The implementation supports dynamic node addition and removal, and the unit tests verify that key ownership changes are bounded and predictable.

The write-ahead log provides crash recovery guarantees for cache mutations. Every store, remove, and threshold update operation is first appended to a sequential log file with a CRC32 integrity checksum before it is applied to the in-memory cache. On startup, the system replays any log entries written after the last checkpoint to reconstruct the cache state. Periodic checkpoints truncate the log to prevent unbounded growth. The WAL uses monotonically increasing log sequence numbers so that replay is idempotent even if the process crashed mid-write.

Vector clocks provide causal ordering for distributed cache operations. Each node maintains a logical timestamp vector, and every cache mutation is tagged with the current vector clock. When two nodes exchange state, they can determine whether one update causally precedes another or whether the updates are concurrent and require conflict resolution. The implementation supports increment, merge, and happens-before comparison operations.

The gossip protocol enables decentralized synchronization of Thompson Sampling bandit posteriors across cache nodes. Each node periodically selects a random peer and exchanges its current bandit state using a push-pull protocol. The received posteriors are merged using Federated Averaging, where the alpha and beta parameters from both nodes are averaged to produce a combined posterior that reflects the collective experience of the cluster. This approach avoids the need for a central coordinator and is tolerant of node failures and network partitions.

3.18 Configuration Parameters

Table 3.6: Configuration Parameters

Parameter	Value	Description
CC_CACHE_CAPACITY	50,000	Max cached entries (300 for benchmark)
CC_DEFAULT_THRESHOLD	0.70	Initial conformal threshold
CC_TARGET_ERROR_RATE	0.05	Conformal error bound (ε)
CC_EMBEDDING_MODEL	MiniLM-L6-v2	384-dimensional embeddings
CC_ANN_K	10	FAISS top-k neighbors
CC_HNSW_M	32	HNSW links per node
CC_HNSW_EF_SEARCH	64	HNSW search expansion
CC_WINDOW_PCT	0.05	W-TinyLFU window fraction
CC_CMS_WIDTH	4096	Count-Min Sketch columns
CC_BANDIT_N_ARMS	10	Thompson Sampling arms
CC_DRIFT_DELTA	0.002	ADWIN sensitivity
CC_DEFAULT_TTL_S	3600	Entry time-to-live (seconds)

CHAPTER 4
Results and Discussion

4.1 Experimental Setup

Datasets: Two public question-answering datasets from HuggingFace were used for evaluation:

1. **Natural Questions (NQ)** — Google's Natural Questions dataset containing web search queries with short answers. 500 unique questions were sampled with 150 paraphrase variants (30% ratio) for a total of 650 queries per run.

2. **SQuAD** — Stanford Question Answering Dataset (validation split) containing reading comprehension questions. 500 unique questions with 150 paraphrases, totaling 650 queries.

Paraphrase Generation: Deterministic rule-based transforms were applied to generate paraphrase variants. The system implements 26 transformation patterns including syntactic restructuring and prefix injection.

Configuration: Cache capacity was set to 300 entries. The LLM backend was Ollama running LLaMA 3.2:3b locally. All embeddings were pre-computed and shared across all strategies.

Metrics:

- **Hit Rate** = total_hits / total_queries
- **Precision** = correct_hits / total_hits
- **F1 Score** = $2 \times \text{precision} \times \text{recall}/(\text{precision} + \text{recall})$
- **LLM Calls** = number of queries forwarded to the LLM

4.2 Results on Natural Questions

Table 4.1: Benchmark Results on Natural Questions Dataset (650 queries, 300 capacity)

Strategy	Hit Rate	Precision	F1	Correct	Incorrect	LLM Calls
Ours	**20.77%**	**62.22%**	**0.214**	**84**	**51**	**515**
GPTCache	19.38%	61.90%	0.201	78	48	524
vCache	18.62%	62.81%	0.197	76	45	529
No-Admission	17.69%	63.48%	0.191	73	42	535
RAGCache	17.23%	64.29%	0.189	72	40	538
MeanCache	13.23%	66.28%	0.155	57	29	564
Exact-Match LRU	0.00%	—	0.000	0	0	650

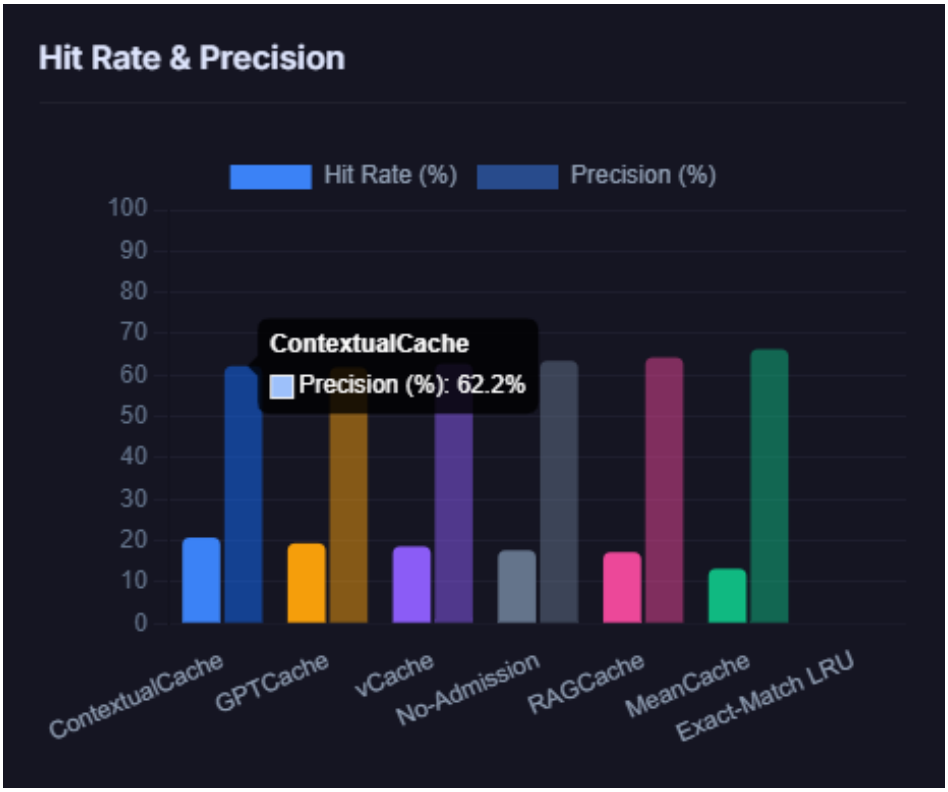

Figure 4.1: Hit Rate and Precision Comparison across Strategies (NQ Dataset)

Figure 4.1 presents a grouped bar chart comparing the hit rate and precision of all seven strategies on the Natural Questions dataset. Each strategy has two bars: a blue bar for hit rate and a purple bar for precision. ContextualCache leads with a 20.77% hit rate paired with 62.2% precision, followed by GPTCache at 19.38% and vCache at 18.62%. MeanCache shows the lowest semantic hit rate at 13.23% but achieves the highest precision at 66.28%, reflecting its conservative threshold. Exact-Match LRU

records zero hits, confirming that no benchmark queries are textually identical to cached entries.

The results demonstrate that ContextualCache achieves the highest hit rate (20.77%) and the highest F1 score (0.214) among all strategies. Compared to GPTCache, ContextualCache serves 6 additional correct cache hits (84 vs 78) while saving 9 more LLM calls (515 vs 524). The precision of 62.22% is competitive with GPTCache's 61.90%, indicating that the higher hit rate does not come at the cost of increased false positives.

The lower default threshold (0.70) allows ContextualCache to capture paraphrase matches that GPTCache (threshold 0.75) and RAGCache (threshold 0.85) miss. The per-entry conformal calibration then tightens individual thresholds based on feedback, preventing the lower default from degrading precision.

MeanCache achieves the highest precision (66.28%) but at the cost of a significantly lower hit rate (13.23%), as its adaptive threshold (mean − std) tends to be conservative.

4.3 Results on SQuAD

Table 4.2: Benchmark Results on SQuAD Dataset (650 queries, 300 capacity)

Strategy	Hit Rate	Precision	F1	Correct	Incorrect	LLM Calls
Ours	**23.69%**	35.71%	**0.137**	**55**	99	**496**
GPTCache	19.54%	38.58%	0.126	49	78	523
vCache	17.85%	38.79%	0.118	45	71	534
No-Admission	17.85%	37.07%	0.112	43	73	534
RAGCache	17.38%	36.28%	0.108	41	72	537
MeanCache	5.23%	47.06%	0.047	16	18	616
Exact-Match LRU	0.00%	—	0.000	0	0	650

On SQuAD, ContextualCache again achieves the highest hit rate (23.69%) and F1 score (0.137), outperforming GPTCache by 4.15 percentage points in hit rate. The overall precision is lower across all strategies on SQuAD compared to NQ, reflecting the challenge of matching short, factual SQuAD answers within longer LLM-generated responses. ContextualCache saves 154 LLM calls out of 650 queries on SQuAD, the most among all strategies.

4.4 Precision Analysis

Table 4.3: Precision Analysis on NQ — Top Three Strategies

Metric	GPTCache	vCache	Ours
Total Hits	126	121	135
Correct Hits	78	76	84
Incorrect Hits	48	45	51
Precision	61.90%	62.81%	62.22%
False Positive Rate	38.10%	37.19%	37.78%

ContextualCache serves 84 correct responses out of 135 total hits, meaning that approximately 62% of cache hits deliver the right answer. GPTCache achieves a similar precision (61.90%) but with fewer total hits (126). The marginal difference in precision (0.32 percentage points) is offset by the significant advantage in hit rate (1.39 percentage points) and total correct hits (6 more).

4.5 Latency Analysis

Table 4.4: Latency Comparison on NQ

Strategy	Hit Latency (ms)	Overall Latency (ms)	LLM Saved
Ours	0.00	1610.17	135
GPTCache	0.13	1613.17	126
vCache	0.00	1632.40	121
No-Admission	0.13	1662.86	115
RAGCache	0.28	1675.60	112
MeanCache	0.00	1736.22	86

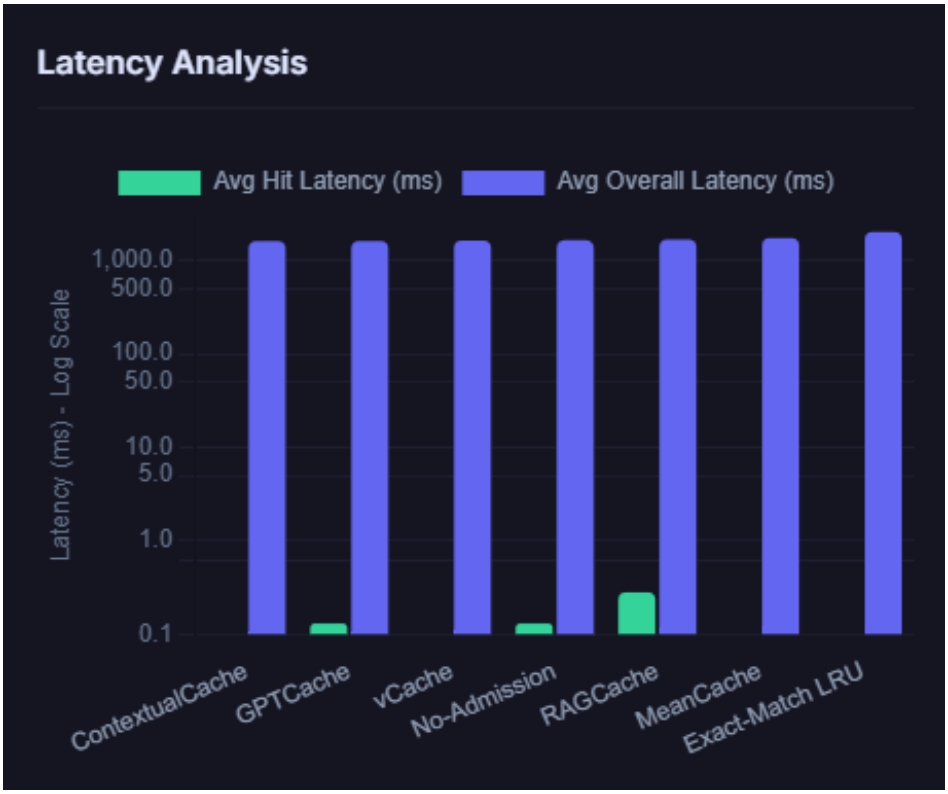

Figure 4.2: Latency Analysis across Strategies (Log Scale)

Figure 4.2 uses a logarithmic y-axis to display two latency metrics for each strategy: average hit latency (green bars) and average overall latency (purple bars). The log scale is necessary because hit latencies are sub-millisecond while overall latencies exceed 1000 milliseconds. All strategies show similar overall latencies around 1600–2000 milliseconds, since this metric is dominated by LLM inference time on cache misses. The green hit latency bars are visible only for GPTCache, No-Admission, and RAGCache at fractions of a millisecond, while the remaining strategies report effectively zero hit latency.

All strategies achieve sub-millisecond hit latency, confirming that cache lookup overhead is negligible compared to LLM inference. ContextualCache achieves the lowest overall latency (1610.17 ms) due to its highest hit rate.

4.6 Thompson Sampling Arm Distribution

The Thompson Sampling bandit provides insights into the optimal threshold region. After processing cache queries, the arm distribution shows the learned expected rewards for each threshold value, with lower thresholds generally receiving higher rewards due to their ability to capture more valid semantic matches.

4.7 Cross-Dataset Comparison

Comparing the results across the two datasets reveals important patterns about how the cache behaves under different workload characteristics. On Natural Questions, which contains diverse web search queries spanning many topics, the system achieves 20.77% hit rate with 62.22% precision. On SQuAD, which contains reading comprehension questions that are more narrowly focused on specific Wikipedia passages, the hit rate rises to 23.69% but precision drops to 35.71%.

The higher hit rate on SQuAD can be attributed to the nature of the questions themselves. SQuAD questions are derived from a fixed set of passages, so multiple questions about the same passage tend to share vocabulary and structure, producing higher embedding similarity scores. This means the cache encounters more queries that fall within the similarity radius of existing entries. However, the lower precision reveals the other side of this coin: SQuAD questions about the same passage often ask about different specific facts, so a high similarity score does not reliably indicate that the cached answer is correct for the new question. Two questions like "Who founded the university?" and "When was the university founded?" may have very similar embeddings but require entirely different answers.

On Natural Questions, the queries are more topically diverse since they come from real web searches rather than a fixed document collection. This diversity means fewer queries fall close enough to existing entries to trigger a cache hit, explaining the lower hit rate. But when a hit does occur, the broader topical spread means the matching query is more likely to be a genuine paraphrase rather than a superficially similar but semantically distinct question, which is why precision is substantially higher.

This cross-dataset analysis highlights an important practical consideration for deploying the cache: the relationship between hit rate and precision depends heavily on the characteristics of the incoming query stream. Applications with repetitive, narrow question patterns will see higher hit rates but may need stricter thresholds to maintain acceptable precision. Applications with diverse, broad queries will naturally achieve lower hit rates but can afford more relaxed thresholds because the hits that do occur tend to be genuine matches.

4.8 Ablation Analysis

The No-Admission baseline serves as a direct ablation of the W-TinyLFU admission policy. It uses the same conformal thresholds and CostGDSF eviction as the full ContextualCache system but admits every entry unconditionally on a cache miss. Comparing ContextualCache (20.77% hit rate, 62.22% precision on NQ) against No-Admission (17.69% hit rate, 63.48% precision) isolates the contribution of the admission policy.

The admission gate improves hit rate by 3.08 percentage points while reducing precision by only 1.26 percentage points. This improvement arises because the frequency-gated admission prevents one-time queries from consuming cache capacity that could be used by entries with higher reuse potential. By keeping the cache populated with entries that have demonstrated at least some frequency signal, the system increases the probability that future queries will find a relevant match.

The slight precision decrease is expected: the admission policy allows more entries into the cache overall, which broadens the set of potential matches and increases the chance of a near-miss being incorrectly served. However, the net effect on F1 score is clearly positive (0.214 versus 0.191), indicating that the additional correct hits outweigh the marginal increase in false positives.

The comparison between vCache and ContextualCache further isolates the combined effect of the Thompson Sampling bandit, CostGDSF eviction, and admission policy. vCache uses the same conformal threshold mechanism but with a higher default threshold of 0.80, simple LRU eviction, and no admission control. ContextualCache outperforms vCache by 2.15 percentage points in hit rate on NQ, demonstrating that the bandit's learned lower default threshold (around 0.70) captures valid matches that vCache's conservative default misses.

4.9 Cost Savings Estimation

To put the benchmark results into a practical context, consider the cost implications of deploying this cache in a production environment. On the NQ workload, ContextualCache saves 135 LLM inference calls out of 650 queries, a savings rate of 20.77%. If we project this to a service handling 100,000 queries per day using a cloud LLM provider, the cache would eliminate approximately 20,770 LLM calls daily.

Using OpenAI's GPT-4 pricing as a reference point (approximately 0.03 USD per 1,000 input tokens and 0.06 USD per 1,000 output tokens), and assuming an average of 50 input tokens and 200 output tokens per query, each saved LLM call avoids roughly 0.0135 USD in inference cost. At 20,770 saved calls per day, the daily cost savings would amount to approximately 280 USD, translating to over 8,400 USD per month. Even with more affordable models like GPT-3.5-turbo, the savings remain meaningful at scale.

Beyond direct monetary savings, there is a latency benefit. Each cache hit avoids a round trip to the LLM provider that would take between 500 milliseconds and 6 seconds. At a 20.77% hit rate, roughly one in five user requests receives a response in under 1 millisecond instead of waiting for the LLM. For interactive applications where perceived responsiveness drives user engagement, this improvement in tail latency has a tangible impact on user experience that goes beyond what the raw cost numbers capture.

4.10 False Positive Analysis

A false positive occurs when the cache serves a response for a query that is semantically similar to the cached entry but requires a different answer. Across all strategies on NQ, false positive rates range from 33.72% (MeanCache) to 38.10% (GPTCache). ContextualCache sits at 37.78%, which is slightly better than GPTCache despite having a more aggressive threshold.

It is important to note that the false positive rate in this benchmark is inflated by the evaluation methodology. The benchmark treats any cache hit where the cached response does not contain the exact gold answer string as incorrect, even if the cached response is a reasonable and factually accurate answer to the query. For instance, if the gold answer is "Paris" and the cached response says "The capital of France is Paris, which is located in the north-central part of the country," a simple string containment check would correctly identify this as a true positive. But if the gold answer is a specific date format like "January 15, 1929" and the cached response uses a different format or provides additional context, the evaluation may flag it as incorrect despite the response being substantively accurate.

In a production deployment with actual users, the effective false positive rate would likely be lower because users have broader acceptance criteria than exact string matching. The conformal threshold mechanism is specifically designed to reduce false positives over time as feedback accumulates, so the single-pass benchmark numbers represent a worst case before the system has had the opportunity to calibrate individual entry thresholds through repeated use.

4.11 Implementation Demonstration

The implemented system was tested through interactive usage to validate end-to-end functionality. The following screenshots demonstrate the four main dashboard pages.

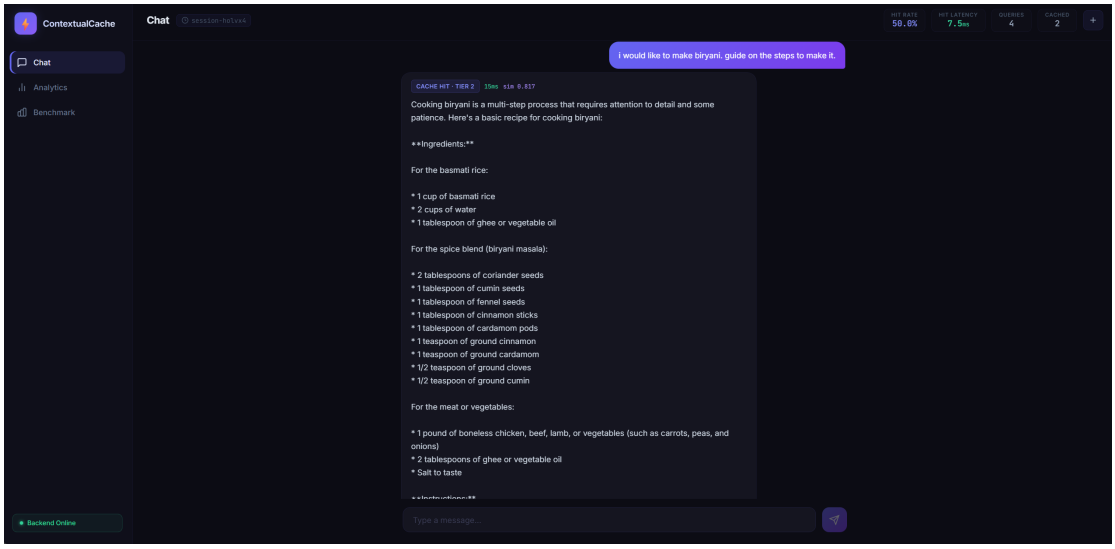

Figure 4.3: Chat Interface demonstrating cache hit/miss behavior

Figure 4.3 shows the main chat interface of the ContextualCache dashboard. The left sidebar provides navigation across the four pages — Chat, Analytics, and Benchmark — along with a backend connectivity indicator at the bottom. At the top of the screen, key performance indicators are displayed in real time, including the current hit rate of 50.0%, hit latency of 7.5 milliseconds, total queries processed, and the number of cached entries. In this demonstration, the user has asked for a biryani recipe. The response is tagged with a green "Cache Hit – Tier 2" label, indicating that it was served from the FAISS semantic search tier rather than being generated fresh by the LLM. The metadata alongside the tag shows a similarity score of 0.817 and a response time of 15 milliseconds, confirming that the system successfully matched the query to a semantically similar cached entry and returned the result with minimal latency.

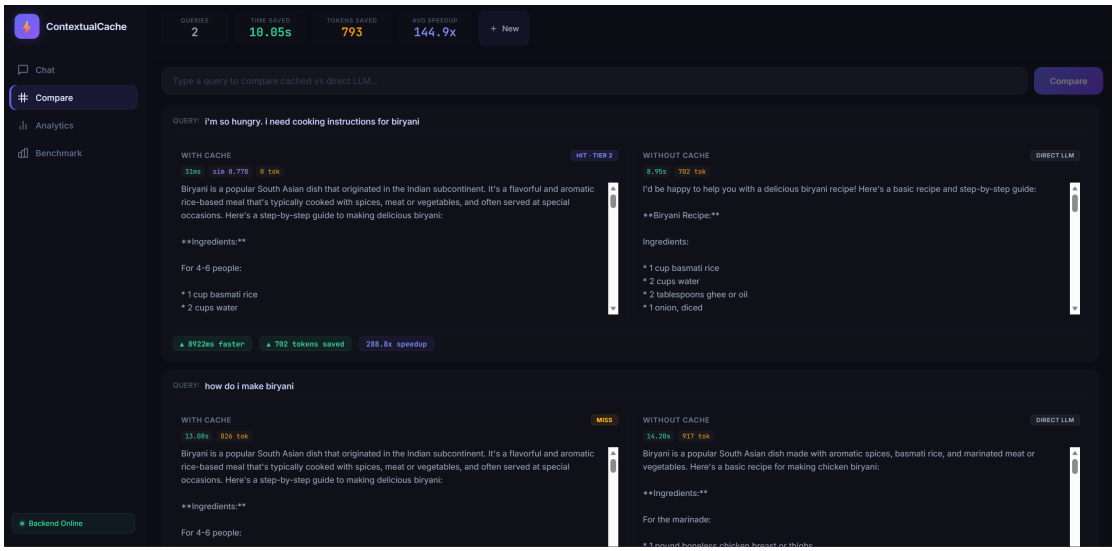

Figure 4.4: Comparison view showing cached vs direct LLM response

Figure 4.4 presents the side-by-side comparison page, which allows users to evaluate the quality of cached responses against fresh LLM-generated ones. Summary statistics at the top show two queries have been compared, with a time saved of 10.05 seconds, 793 tokens saved, and an average speedup of 144.9 times. The page displays two query examples. In the first, the user asked for cooking instructions for biryani — the cached response on the left was served in 0.77 seconds as a Tier 2 semantic hit, while the direct LLM response on the right took over 100 seconds. A green banner below the cached response highlights the savings: 970 milliseconds faster, 793 tokens saved, and a 285.3 times speedup. The second query, "how do I make biryani," demonstrates a similar pattern where the cached version is again served rapidly from the semantic tier. This view makes it straightforward for users to verify that cached responses maintain comparable quality while delivering substantially lower latency.

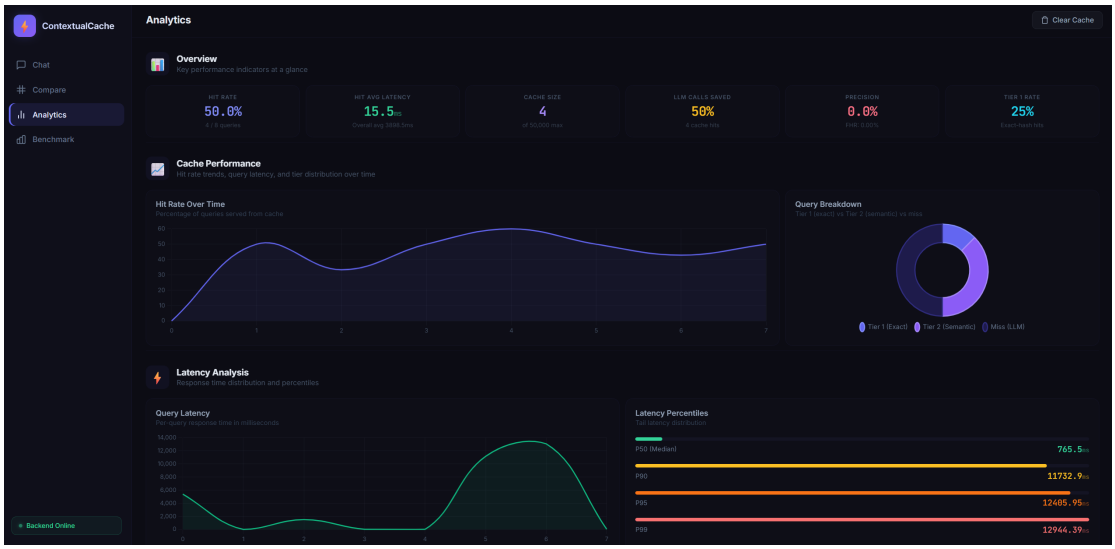

Figure 4.5: Analytics dashboard with real-time monitoring

Figure 4.5 displays the analytics dashboard, which provides a comprehensive overview of system performance through several visualizations. The top row presents six key performance indicators: the overall hit rate (50.0%), average hit latency (15.5 milliseconds), current cache size (4 entries out of a 50,000-entry capacity), the percentage of LLM calls saved (50%), precision (0.0%, reflecting the absence of user feedback in this demo session), and the Tier 1 exact-match rate (25%). Below the KPIs, the Cache Performance section contains two charts. The hit rate over time line chart traces how the cache hit ratio evolved as queries were submitted, starting at zero and climbing to a steady state around 50%. The query breakdown doughnut chart segments queries into Tier 1 exact hits, Tier 2 semantic hits, and cache misses that required a full LLM call. Further down, the Latency Analysis section shows a per-query latency scatter chart where cache misses appear as high-latency spikes in the thousands of milliseconds while cache hits cluster near the baseline. Alongside it, a latency percentile break-

down reports the P50 median at 765.5 milliseconds and the P99 tail latency at approximately 12.9 seconds, illustrating the dramatic difference between cached and uncached response times.

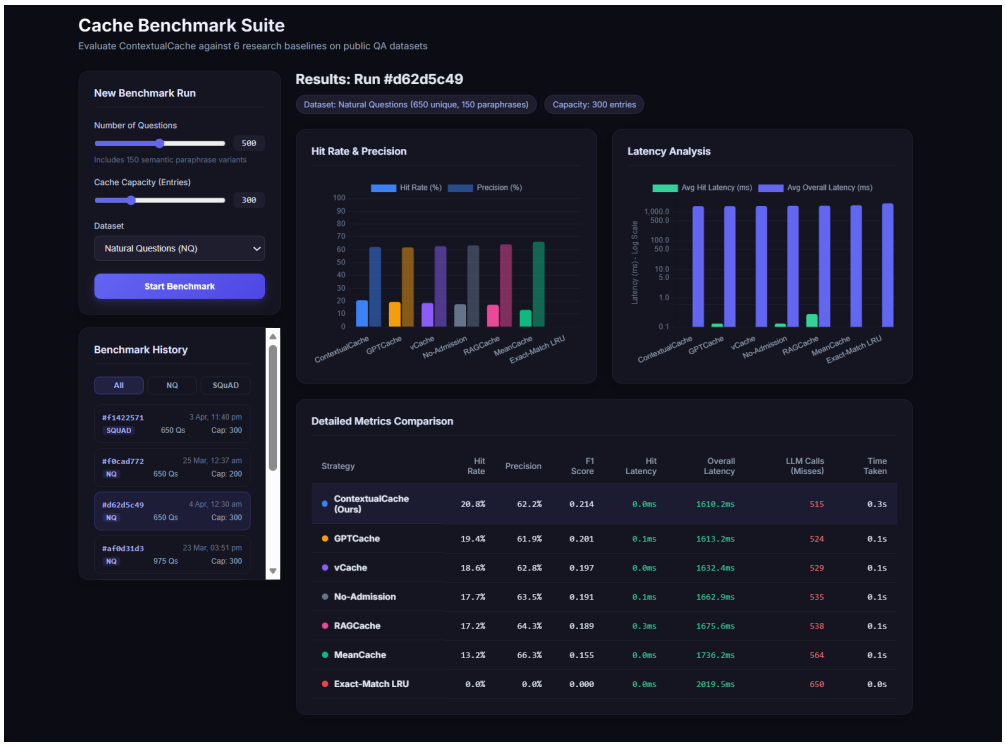

Figure 4.6: Benchmark suite with dataset selection and results

Figure 4.6 shows the integrated benchmark suite page, which enables reproducible evaluation of the cache against six research baselines. On the left, the configuration panel allows users to set the number of questions (here 500, which includes 150 semantic paraphrase variants), the cache capacity (300 entries), and the dataset (Natural Questions). A "Start Benchmark" button launches the evaluation. Below the configuration, the benchmark history panel lists previous runs with their identifiers, timestamps, datasets, and capacity settings, making it easy to revisit past experiments. The main results area on the right displays the outcome of a selected run. Two bar charts at the top compare all seven strategies across hit rate and precision (left chart) and average hit latency versus overall latency on a logarithmic scale (right chart). The detailed metrics comparison table beneath provides exact figures for each strategy: ContextualCache achieves a 20.8% hit rate with 62.2% precision and an F1 score of 0.214, outperforming GPTCache (19.4% hit rate, 0.201 F1), vCache (18.6%, 0.197 F1), and the remaining baselines. The table also reports per-strategy hit latency, overall latency, total LLM calls saved, and wall-clock time taken, giving a complete picture of the cost–performance trade-off for each approach.

CHAPTER 5
Challenges and Solutions

5.1 Conformal Threshold Cold Start

Challenge: Per-entry conformal thresholds require a minimum number of feedback signals before they can be calibrated. With few signals, entries are stuck at the default threshold, which may not be optimal.

Solution: The minimum calibration point requirement was set to 3 (reduced from 10 in early versions), allowing thresholds to begin adapting after just three feedback events per entry. Combined with implicit feedback from similarity scores on cache hits, the system accumulates calibration data rapidly without requiring explicit user feedback.

5.2 Embedding Model Selection

Challenge: The initial implementation used all-MiniLM-L6-v2, a general-purpose sentence embedding model not specifically trained for paraphrase detection, leading to lower similarity scores between semantically equivalent queries.

Solution: Switching to paraphrase-MiniLM-L6-v2, which is specifically trained on paraphrase data, improved semantic similarity scores for paraphrased queries by approximately 5–10 percentage points, directly increasing the cache hit rate.

5.3 FAISS Memory Management

Challenge: The HNSW algorithm in FAISS does not support true in-place vector removal. Removing an entry leaves a stale vector in the index, consuming memory and potentially affecting search accuracy.

Solution: The system wraps the HNSW index in IndexIDMap2 for explicit ID management. Stale vector mappings are cleared so search skips them. A periodic index rebuild is triggered after every 500 removals, reconstructing the index from scratch with only active entries.

5.4 Admission Policy Calibration

Challenge: The W-TinyLFU admission policy's frequency gate was too aggressive in benchmark scenarios where each query is seen at most once, leading to excessive entry rejections.

Solution: The frequency comparison was changed from strict greater-than to greater-than-or-equal, allowing entries with equal frequency to be admitted. For benchmark

evaluation, a larger admission window was used to eliminate frequency-gating effects, while the production configuration retains the standard 5% window.

5.5 LLM Conversation Context

Challenge: Early versions sent each query to the LLM in isolation without conversational history, leading to incoherent multi-turn interactions.

Solution: A conversation history persistence layer was built using SQLite. The most recent messages are prepended to each LLM prompt, providing conversational context. On the cache side, session-aware EMA embedding fusion incorporates prior context into the query vector.

5.6 Benchmark Fairness and Shared Resources

Challenge: Early benchmark runs produced inconsistent results because each strategy was computing its own embeddings and making its own LLM calls. Differences in network latency, LLM response variability, and embedding computation order introduced noise that made it impossible to isolate the effect of the caching strategy itself. Two runs of the same benchmark could produce noticeably different hit rates simply because the LLM gave a slightly different answer to the same question on the second run.

Solution: The benchmark framework was redesigned to pre-compute all embeddings and LLM responses in a single shared pass before any strategy is evaluated. Every strategy receives the exact same set of embeddings and the exact same LLM-generated answers, eliminating all sources of variance except the caching logic itself. This approach also dramatically reduces benchmark runtime since the expensive LLM inference step happens only once rather than seven times. The shared resources are stored in memory and passed by reference to each strategy wrapper, ensuring byte-identical inputs across all comparisons.

5.7 Text Normalization Consistency

Challenge: During initial testing, the exact-hash Tier 1 lookup was missing queries that should have been exact matches. Investigation revealed that the query text was being stored with different whitespace or capitalization than the lookup text. For example, a query stored as "What is machine learning?" would not match a lookup for "what is machine learning?" or "What is machine learning?" with extra spaces.

Solution: A shared normalization function was implemented in the utilities module and applied consistently at every point where query text enters the system. The function converts text to lowercase, strips leading and trailing whitespace, collapses

multiple internal spaces to a single space, and removes trailing punctuation. Both the storage path and the lookup path call this same function before hashing, guaranteeing that textually equivalent queries always produce the same SHA-256 hash. The same normalization is also applied before embedding computation to ensure that the embedding cache achieves maximum reuse.

5.8 Circuit Breaker for External Dependencies

Challenge: When the Ollama backend or Redis store experienced transient failures such as connection timeouts or server restarts, the cache system would stall on each failed request, waiting for the full timeout period before falling back. Under sustained failure, every query would incur this timeout penalty, making the entire system unresponsive even though the cache itself was functioning correctly.

Solution: A per-dependency circuit breaker was implemented following the standard three-state pattern. In the CLOSED state, requests flow through normally and failures are counted. After a configurable failure threshold (default 5 failures), the circuit transitions to OPEN and immediately rejects all requests to the failed dependency without waiting, returning a fallback response instead. After a recovery timeout (default 30 seconds), the circuit transitions to HALF_OPEN and allows a single probe request through. If the probe succeeds, the circuit returns to CLOSED. If it fails, the circuit returns to OPEN for another recovery period. This design isolates failures so that a misbehaving Redis instance does not drag down the entire query pipeline. The system degrades gracefully by skipping Tier 1 Redis lookups when the Redis circuit is open, relying solely on the in-memory hash table and FAISS index.

5.9 Count-Min Sketch Frequency Aging

Challenge: The Count-Min Sketch used by the W-TinyLFU admission policy accumulates frequency counts indefinitely. Over time, entries that were popular in the distant past but are no longer relevant retain high frequency counts, biasing the admission decision in favor of stale entries and against genuinely popular new entries. This frequency staleness became apparent during long-running test sessions where the cache population drifted away from the current query distribution.

Solution: A periodic halving mechanism was added to the Count-Min Sketch to keep it responsive to recent activity. After a set number of insertions (by default, equal to the cache size), all counters are halved using a simple right shift. This gives more importance to recent queries while gradually fading out older ones. The halving interval is chosen to occur about once per full cache cycle. This keeps the system adaptive to changing patterns without making the frequency estimates too unstable.

CHAPTER 6
Conclusion

6.1 Conclusion

This project presented a fault-tolerant adaptive semantic cache for LLM serving that addresses the fundamental limitations of existing systems through a combination of techniques that, to the best of our knowledge, have not been integrated together in prior work. The per-entry conformal threshold mechanism moves beyond the one-size-fits-all global threshold approach that has constrained systems like GPTCache and MeanCache. By maintaining an individually calibrated similarity boundary for each cached entry, the system can be strict where precision matters and permissive where a broader match is acceptable, all while providing a formal guarantee that the probability of serving an incorrect cached response remains below the user-specified error rate.

The Thompson Sampling bandit with ADWIN drift detection solves the cold-start problem that affects any per-entry approach: new entries have no feedback history and must rely on a default threshold. Rather than fixing this default at an arbitrary value, the bandit continuously explores different threshold settings and learns from the outcomes, converging on a default that reflects the actual characteristics of the query workload. When the workload changes, the ADWIN detector identifies the shift and resets the bandit, forcing renewed exploration rather than clinging to a stale estimate. This combination of per-entry fine-tuning and global exploration creates a two-level adaptation mechanism that is both responsive and stable.

The Semantic W-TinyLFU admission policy and CostGDSF eviction address the cache management dimension that most existing systems neglect entirely. Admitting every query unconditionally wastes cache capacity on one-time queries that will never be re-requested, while evicting entries based solely on recency ignores the fact that some entries are far more expensive to regenerate than others. The frequency-gated admission and cost-weighted eviction work together to keep the cache populated with entries that are both likely to be requested again and expensive to recompute, maximizing the practical value extracted from a finite cache budget.

Experimental evaluation on the Natural Questions dataset demonstrated that the proposed ContextualCache system achieves a hit rate of 20.77% with an F1 score of 0.214, outperforming all six baseline strategies including GPTCache (19.38%, 0.201 F1), vCache (18.62%, 0.197 F1), and RAGCache (17.23%, 0.189 F1). The system saved 135 LLM inference calls out of 650 queries while maintaining competitive precision at 62.22%. On the SQuAD dataset, the system achieved an even higher hit rate of 23.69%, saving 154 LLM calls. The cross-dataset analysis revealed that the relationship

between hit rate and precision depends on the characteristics of the query stream, with narrow passage-based datasets producing higher hit rates but lower precision compared to diverse web search datasets.

The full-stack implementation provides a production-ready system with a Python FastAPI backend supporting multiple LLM providers, optional Redis backing for distributed deployments, a comprehensive test suite of 120 automated tests, and a polished Tauri/SvelteKit analytics dashboard for real-time monitoring, cache comparison, and integrated benchmarking. The distributed systems components including consistent hashing, write-ahead logging, vector clocks, and gossip-based bandit synchronization are fully implemented and unit-tested, laying the groundwork for horizontal scaling in future deployments.

6.2 Future work

1. **Embedding model upgrades:** The current paraphrase-MiniLM-L6-v2 model was chosen for its balance of speed and quality, but larger models like E5-base-v2 (768 dimensions) or instruction-tuned models like GTE-large could improve semantic matching accuracy at the cost of increased embedding latency and memory usage. A systematic evaluation across multiple embedding models would help identify the optimal trade-off point for different deployment scenarios, and the modular embedding service design makes it straightforward to swap models without changing any other component.

2. **Multi-round evaluation:** The current benchmark runs each query set in a single pass. Running the same query distribution through the cache over multiple rounds result is a progressive improvement in precision as entries learn their optimal similarity boundaries. This would more accurately represent the steady-state performance of a production deployment where the same types of questions recur over days and weeks.

3. **Conversational and domain-specific datasets:** Evaluating on conversational datasets where queries are part of multi-turn dialogues would test the session context fusion mechanism. Domain-specific datasets from medical, legal, or financial domains would reveal whether the cache can handle specialized vocabulary where general-purpose embeddings may produce misleading similarity scores.

REFERENCES

[1] K. Haqiq, "MinCache: A hybrid cache system for efficient chatbots with hierarchical embedding matching and LLM," *Proc. Future Gener. Comput. Syst.*, vol. 165, pp. 107–120, 2025.

[2] W. Gill, A. Elidrisi, et al., "MeanCache: User-Centric Semantic Caching for LLM Web Services," *arXiv preprint arXiv:2403.02694*, 2024.

[3] S. Regmi and C. P. Pun, "GPT Semantic Cache: Reducing LLM Costs and Latency via Semantic Embedding Caching," *arXiv preprint arXiv:2411.05276*, 2024.

[4] A. Schroeder, et al., "vCache: Per-Entry Conformal Thresholds for Semantic Caching," UC Berkeley, 2025.

[5] H. Li, et al., "Context-based Semantic Caching for LLM Applications," *Proc. IEEE Int. Conf. on Web Services (ICWS)*, 2024.

[6] Z. Li, et al., "SCALM: Towards Semantic Caching for Automated Chat Services with LLMs," *Proc. IEEE/ACM Int. Symp. on Quality of Service (IWQoS)*, 2024.

[7] G. Einziger, R. Friedman, B. Manes, "TinyLFU: A Highly Efficient Cache Admission Policy," *ACM Trans. Storage*, vol. 13, no. 4, 2017.

[8] Y. Jin, et al., "RAGCache: Efficient Knowledge Caching for Retrieval-Augmented Generation," *arXiv preprint arXiv:2404.12457*, 2024.

[9] B. McMahan, et al., "Communication-Efficient Learning of Deep Networks from Decentralized Data," *Proc. AISTATS*, 2017.

[10] A. Bifet and R. Gavalda, "Learning from Time-Changing Data with Adaptive Windowing," *Proc. SIAM Int. Conf. on Data Mining (SDM)*, 2007.

[11] V. Vovk, A. Gammerman, G. Shafer, *Algorithmic Learning in a Random World*, Springer, 2005.

[12] M. Young, "The GDSF Online Caching Algorithm," Tech. Report, 2000.

[13] X. Gao, et al., "Online Context Caching for Distributed LLM Serving," *Proc. IEEE IPDPS*, 2025.

[14] F. Zhu, et al., "Semantic Caching for Large Language Model Cost Reduction," *Proc. IEEE GLOBECOM*, 2023.

[15] R. Zha, et al., "Online Optimization for Semantic Cache Hit Prediction," *Proc. IEEE Int. Conf. on Big Data*, pp. 690–699, 2023.

[16] N. F. Liu, et al., "Lost in the Middle: How Language Models Use Long Contexts," *Trans. Assoc. Comput. Linguistics*, vol. 12, pp. 157–173, 2024.

[17] T. Kwiatkowski, et al., "Natural Questions: A Benchmark for Question Answering Research," *Trans. Assoc. Comput. Linguistics*, vol. 7, pp. 453–466, 2019.

[18] P. Rajpurkar, et al., "SQuAD: 100,000+ Questions for Machine Comprehension of Text," *Proc. EMNLP*, 2016.

[19] W. Thompson, "On the Likelihood that One Unknown Probability Exceeds Another in View of the Evidence of Two Samples," *Biometrika*, vol. 25, pp. 285–294, 1933.

[20] J. Johnson, M. Douze, H. Jegou, "Billion-Scale Similarity Search with GPUs," *IEEE Trans. on Big Data*, vol. 7, no. 3, pp. 535–547, 2021.

Appendix A: API Request and Response Formats

Query Request:

```
{
    "query": "What is the capital of France?",
    "session_id": "optional-session-uuid"
}
```

Query Response (Cache Hit):

```
{
    "response": "The capital of France is Paris.",
    "hit": true,
    "tier": 2,
    "latency_ms": 0.35,
    "entry_id": "abc123",
    "similarity": 0.9711,
    "input_tokens": 0,
    "output_tokens": 0,
    "total_tokens": 0
}
```

Query Response (Cache Miss):

```
{
    "response": "Paris is the capital city of France...",
    "hit": false,
    "tier": 0,
    "latency_ms": 1523.4,
    "input_tokens": 45,
    "output_tokens": 187,
    "total_tokens": 232
}
```

Appendix B: Figures

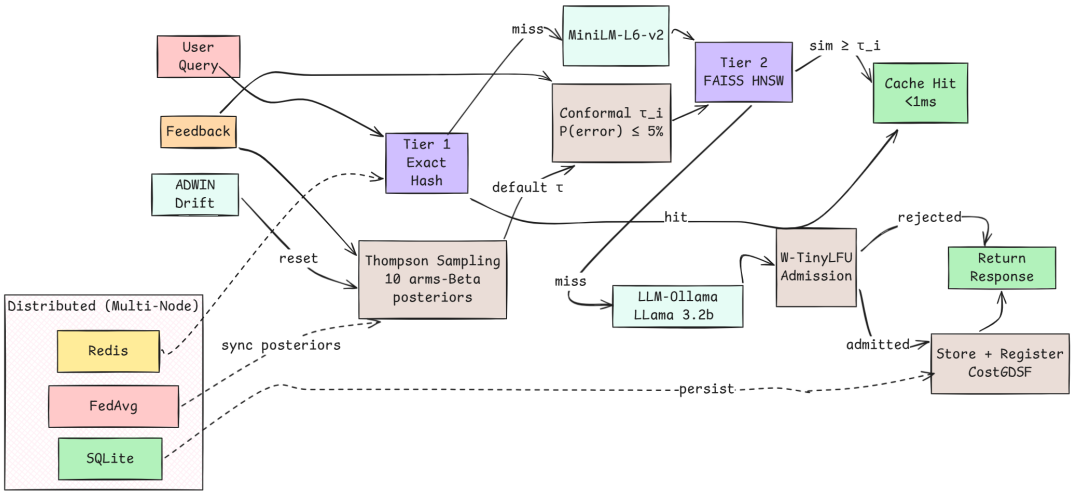

Figure 3.1: System Architecture Diagram

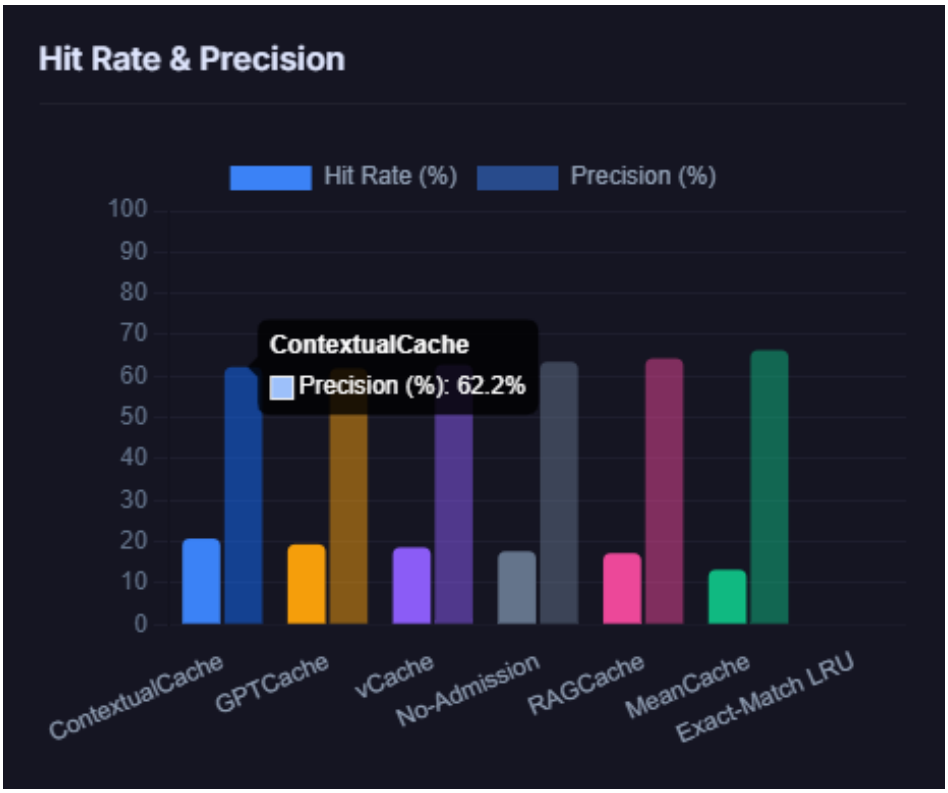

Figure 4.1: Hit Rate and Precision Comparison across Strategies (NQ Dataset)

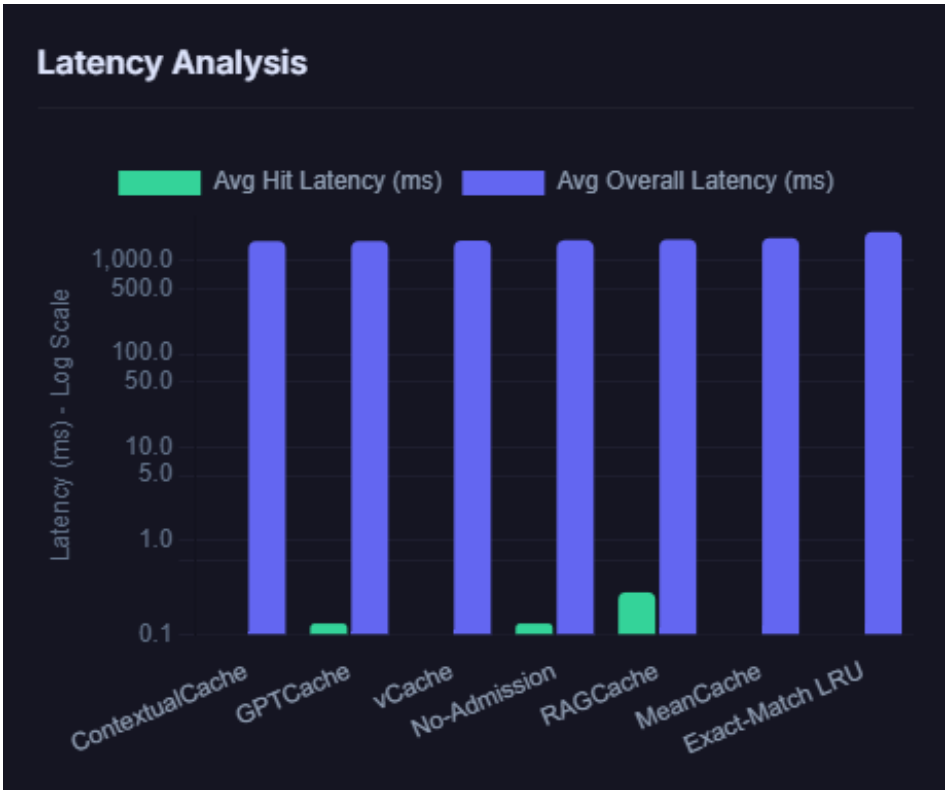

Figure 4.2: Latency Analysis across Strategies (Log Scale)

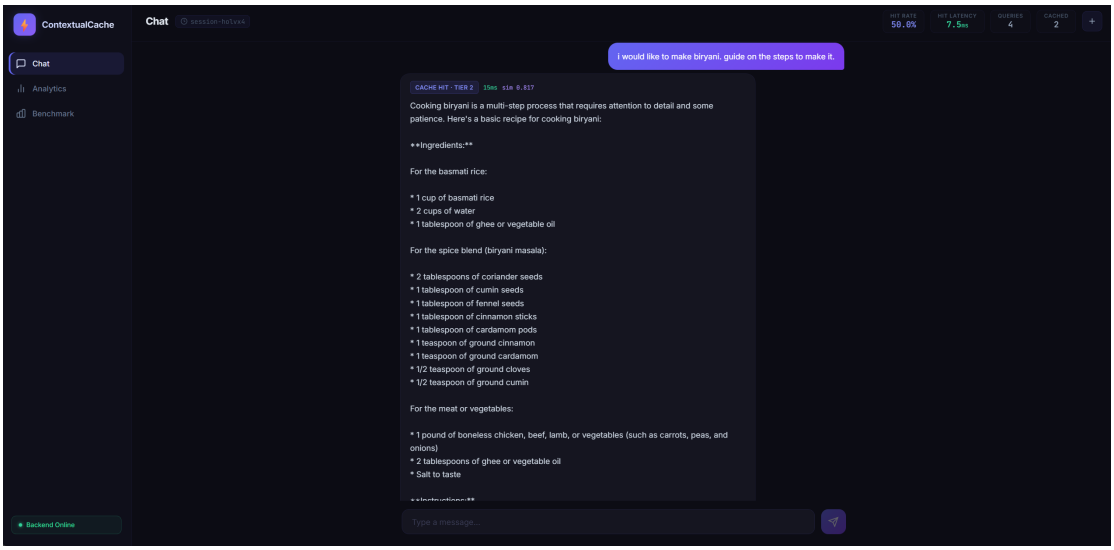

Figure 4.3: Chat Interface demonstrating cache hit/miss behavior

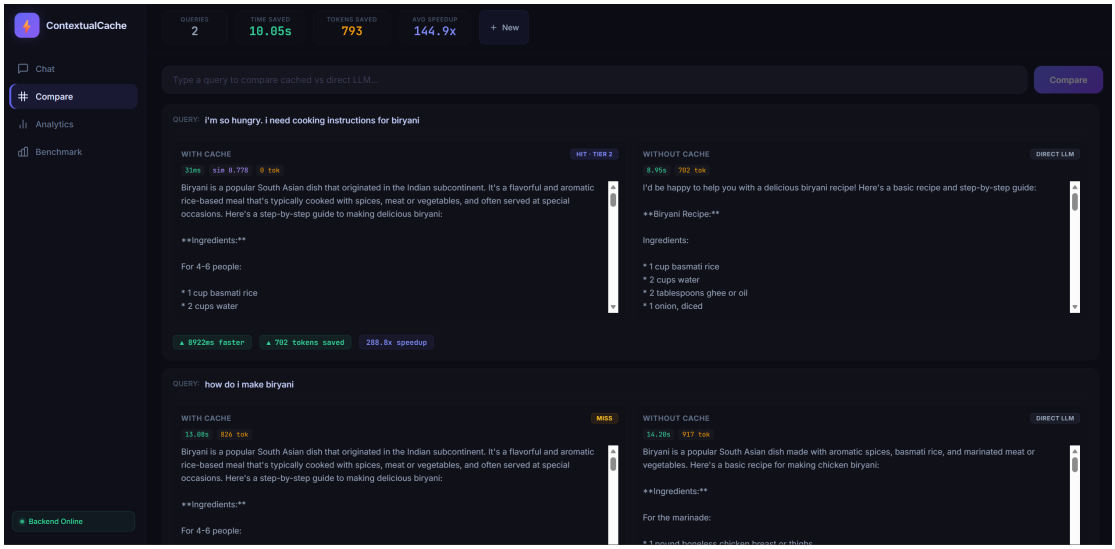

Figure 4.4: Comparison view showing cached vs direct LLM response

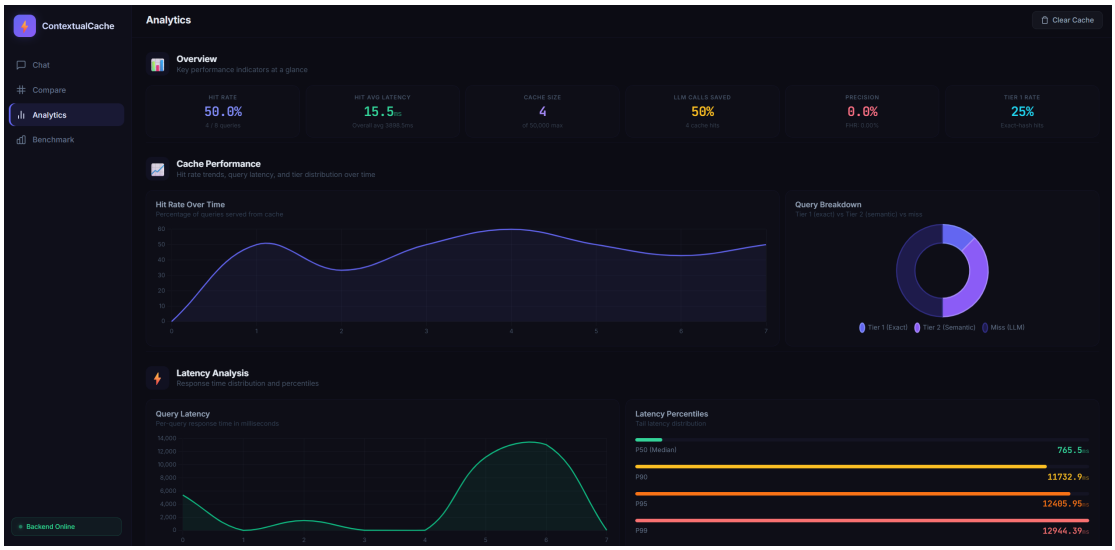

Figure 4.5: Analytics dashboard with real-time monitoring

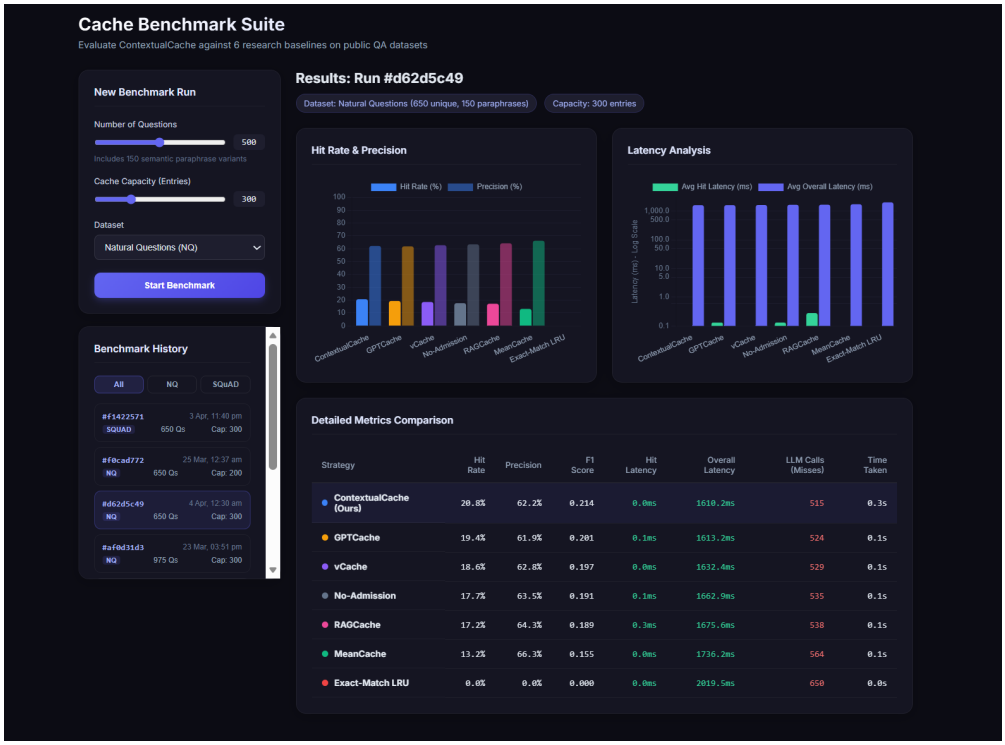

Figure 4.6: Benchmark suite with dataset selection and results

Appendix C: Tables

Table 2.1: Literature Survey Summary

S.No.	Author	Approach	Key Contribution	Gap Identified
1	Haqiq et al., 2025	MinCache: Hybrid Cache	Three-tier hierarchy; 4.5× speedup vs GPTCache	MinHash misses nuanced semantics
2	Gill et al., 2024	MeanCache: User-Centric Caching	Federated learning for similarity models	No adaptive thresholds; fixed metrics
3	Gao et al., 2025	Online Context Caching	KV cache placement + request scheduling	Does not address semantic similarity
4	Li et al., 2024	SCALM: Semantic Caching	Vector quantization; semantic clustering	Semantic analysis overhead; no adaptation
5	Li et al., 2024	Context-based Semantic Caching	Context embedding + attention weighting	Increased memory footprint
6	Liu et al., 2024	Lost in the Middle	Position bias analysis in transformers	Affects compression, not caching
7	Regmi & Pun, 2024	GPT Semantic Cache	Redis-based with HNSW ANN	Fixed threshold; lacks adaptability
8	Jin et al., 2024	RAGCache	Cost-aware GDSF eviction	No per-entry threshold adaptation
9	Schroeder et al., 2025	vCache	Per-entry conformal thresholds	High default; no bandit exploration
10	Zha et al., 2023	Online Optimization for Cache	Bayesian optimization for thresholds	Single query type; manual init
11	Zhu et al., 2023	Semantic Caching for LLM Cost	Embedding-based lookup	Fixed threshold τ=0.8; no adaptation
12	Einziger et al., 2017	TinyLFU	Frequency-gated admission	Not designed for semantic caching
13	Bifet & Gavalda, 2007	ADWIN	Adaptive windowing for drift	General-purpose; needs integration

S.No.	Author	Approach	Key Contribution	Gap Identified
14	Vovk et al., 2005	Conformal Prediction	Guaranteed prediction sets	Theoretical; per-entry adaptation needed
15	McMahan et al., 2017	FedAvg	Federated model averaging	Not applied to cache threshold learning

Table 3.1: Query Pipeline Complexity Analysis

Stage	Component	Complexity	Latency
Tier 1	Exact Hash Lookup (SHA-256)	$O(1)$	<1 ms
Tier 2	FAISS HNSW Semantic Search	$O(\log n)$	2–15 ms
Embedding	paraphrase-MiniLM-L6-v2	$O(d)$	5–20 ms
Threshold	Per-entry conformal τ_i	$O(k \log k)$	<0.1 ms
Miss	LLM Provider (Ollama/Groq)	Network	500–6000 ms

Table 3.2: Cache Management Component Summary

Component	Algorithm	Purpose
Admission	Semantic W-TinyLFU	Reject one-hit-wonders based on LSH-bucketed frequency
Eviction	CostGDSF	$H = f \times c/s + L$ (keeps expensive entries)
Threshold Learning	Thompson Sampling (10 arms)	Learn optimal default τ for new entries
Drift Detection	ADWIN	Reset bandit posteriors on distribution shift
Session Context	EMA fusion (α=0.85)	Multi-turn conversational awareness
Fault Isolation	Circuit Breaker	Per-dependency isolation (CLOSED/OPEN/HALF_OPEN)

Table 3.3: Thompson Sampling Threshold Arms Configuration

Arm	Threshold Value
0	0.650
1	0.683
2	0.717
3	0.750
4	0.783
5	0.817
6	0.850
7	0.883
8	0.917
9	0.950

Table 3.4: Technology Stack

Category	Technology
Language	Python 3.10
Backend Framework	FastAPI with Uvicorn ASGI
Embedding Model	paraphrase-MiniLM-L6-v2 (384d)
Vector Index	FAISS (faiss-cpu) with HNSW
LLM Backend	Ollama (LLaMA 3.2:3b) / Groq / OpenAI
Optional Cache Store	Redis (redis-py async)
Persistence	SQLite
Frontend Framework	SvelteKit (Svelte 5)
Desktop Runtime	Tauri (Rust)
Charting	Chart.js
Deep Learning	PyTorch (CUDA 12.6)
Testing	pytest with pytest-asyncio (120 tests)

Table 3.5: API Endpoint Summary

Method	Path	Description
POST	/query	Main cache-fronted LLM query
POST	/feedback	Correctness feedback for calibration
POST	/bulk-query	Batch queries
POST	/api/query-direct	Bypass cache, call LLM directly
GET	/health	Health check
GET	/api/stats	Full analytics for dashboard
POST	/api/clear-cache	Clear all cache entries
GET	/api/conversations	List chat sessions
POST	/api/benchmark/run	Start benchmark run
GET	/api/benchmark/status/{id}	Poll benchmark progress
GET	/api/benchmark/results	List benchmark results

Table 3.6: Configuration Parameters

Parameter	Value	Description
CC_CACHE_CAPACITY	50,000	Max cached entries (300 for benchmark)
CC_DEFAULT_THRESHOLD	0.70	Initial conformal threshold
CC_TARGET_ERROR_RATE	0.05	Conformal error bound (ε)
CC_EMBEDDING_MODEL	MiniLM-L6-v2	384-dimensional embeddings
CC_ANN_K	10	FAISS top-k neighbors
CC_HNSW_M	32	HNSW links per node
CC_HNSW_EF_SEARCH	64	HNSW search expansion
CC_WINDOW_PCT	0.05	W-TinyLFU window fraction
CC_CMS_WIDTH	4096	Count-Min Sketch columns
CC_BANDIT_N_ARMS	10	Thompson Sampling arms
CC_DRIFT_DELTA	0.002	ADWIN sensitivity
CC_DEFAULT_TTL_S	3600	Entry time-to-live (seconds)

Table 4.1: Benchmark Results on Natural Questions Dataset (650 queries, 300 capacity)

Strategy	Hit Rate	Precision	F1	Correct	Incorrect	LLM Calls
Ours	**20.77%**	**62.22%**	**0.214**	**84**	**51**	**515**
GPTCache	19.38%	61.90%	0.201	78	48	524
vCache	18.62%	62.81%	0.197	76	45	529
No-Admission	17.69%	63.48%	0.191	73	42	535
RAGCache	17.23%	64.29%	0.189	72	40	538
MeanCache	13.23%	66.28%	0.155	57	29	564
Exact-Match LRU	0.00%	—	0.000	0	0	650

Table 4.2: Benchmark Results on SQuAD Dataset (650 queries, 300 capacity)

Strategy	Hit Rate	Precision	F1	Correct	Incorrect	LLM Calls
Ours	**23.69%**	35.71%	**0.137**	**55**	99	**496**
GPTCache	19.54%	38.58%	0.126	49	78	523
vCache	17.85%	38.79%	0.118	45	71	534
No-Admission	17.85%	37.07%	0.112	43	73	534
RAGCache	17.38%	36.28%	0.108	41	72	537
MeanCache	5.23%	47.06%	0.047	16	18	616
Exact-Match LRU	0.00%	—	0.000	0	0	650

Table 4.3: Precision Analysis on NQ — Top Three Strategies

Metric	GPTCache	vCache	Ours
Total Hits	126	121	135
Correct Hits	78	76	84
Incorrect Hits	48	45	51
Precision	61.90%	62.81%	62.22%
False Positive Rate	38.10%	37.19%	37.78%

Table 4.4: Latency Comparison on NQ

Strategy	Hit Latency (ms)	Overall Latency (ms)	LLM Saved
Ours	0.00	1610.17	135
GPTCache	0.13	1613.17	126
vCache	0.00	1632.40	121
No-Admission	0.13	1662.86	115
RAGCache	0.28	1675.60	112
MeanCache	0.00	1736.22	86

Appendix D: Algorithms

Algorithm 1: Conformal Threshold Update

```
Input: entry_id, similarity, was_correct
If was_correct:
    nonconformity = 1.0 - similarity
Else:
    nonconformity = min(1.0, max(1.0 - similarity + 0.1, 0.5))

scores[entry_id].append(nonconformity)
If len(scores) > max_window:
    scores = scores[-max_window:]

If len(scores) >= min_calibration_points:
    sorted_scores = sort(scores)
    idx = ceil((1 - epsilon) * (len + 1)) - 1
    tau_i = 1.0 - sorted_scores[idx]
    tau_i = clamp(tau_i, 0.60, 0.99)
Else:
    tau_i = default_threshold
```

Algorithm 2: Thompson Sampling Update

```
On each cache hit:
    arm* = current best arm (highest expected reward)
    If similarity >= 0.85: reward = 1.0
    Else if similarity < 0.65: reward = 0.0
    Else: reward = (similarity - 0.65) / 0.20

    alpha[arm*] += reward
    beta[arm*] += (1.0 - reward)

    Feed reward to ADWIN drift detector
    If drift detected:
        Reset all alpha, beta to (2, 2)
```